Program Analysis 2025-26

Lecture #13

Galois insertions

Università di Pisa

Abstract Interpretation

Abstract Interpretation

It is a technique to formally reason on approximations

It allows to derive effective methods to compute approximations
Generally used to compute overapproximations
Seldom used to compute underapproximations

“Can Abstract Interpretation play a guiding and explanatory role for a wide range of static and dynamic under-approximate tools for bug catching, similar to what it already does for over-approximate analyses?”

Example: out of bounds

```
function arrayOutOfBounds(int n, int x[10]) {

  a = 0

  if n >= 10 then

     n = n - 5

  else

     a = ++n


  a = max(0,a – n)

  return x[a] }
```

Let us assume $n \geq 0$

Is it a safe access? ($0 \leq a \leq 9$?)

Using exact semantics

```
function arrayOutOfBounds(int n, int x[10]) {
(0,_)(1,_)(2,_)(3,_)(4,_)(5,_)(6,_)(7,_)(8,_)(9,_)(10,_)…
  a = 0
(0,0)(1,0)(2,0)(3,0)(4,0)(5,0)(6,0)(7,0)(8,0)(9,0)(10,0)…
  if n >= 10 then
(10,0)(11,0)(12,0)(13,0)(14,0)(15,0)(16,0)(17,0)(18,0)(19,0)…
     n = n - 5
(5,0)(6,0)(7,0)(8,0)(9,0)(10,0)(11,0)(12,0)(13,0)(14,0)…
  else

     a = ++n


  a = max(0,a – n)

  return x[a] }
```

We can't track infinite set of pairs!

let's use intervals !

memory = (value of n,value of a)

Abstract Interpretation: the idea

Goal: Compute the set S of possible values at each line of code

But… this is not feasible in general

We compute an (over)approximation $S^{\#} \supseteq S$

The theory of abstract interpretation allows to compute $S^{\#}$ as a set of abstract values obtained by applying abstract operations in a fixed abstract domain

Example: interval abstraction

```
 function arrayOutOfBounds(int n, int x[10]) {
[0,∞][-∞,∞]                abstract memory = interval of n interval of a
    a = 0                  (this is called a non-relational abstraction)
[0,∞][0,0]
    if n >= 10 then
     [10,∞][0,0]
        n = n - 5
     [5,∞][0,0]
    else
     [0,9][0,0]
        a = ++n
     [1,10][1,10]
[1,∞][0,10]
    a = max(0,a – n)
[1,∞][0,9]
    return x[a] }
```

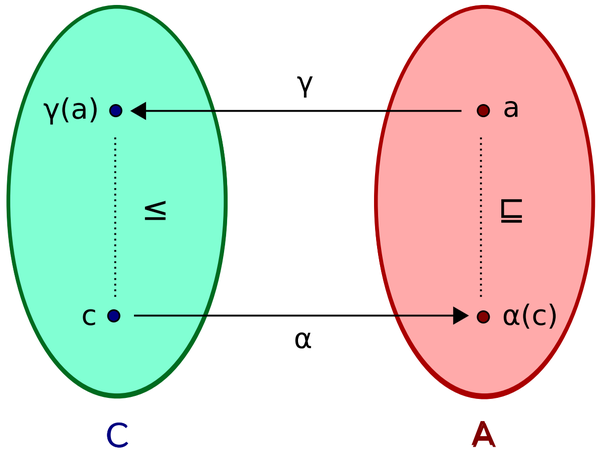
Merging branches looses precision:

$[1,\infty] = [5,\infty] \cup [1,10]$

$[0,10] = [0,0] \cup [1,10]$

safe! $0 \leq a \leq 9$!

Ingredients of Abstract Interpretation

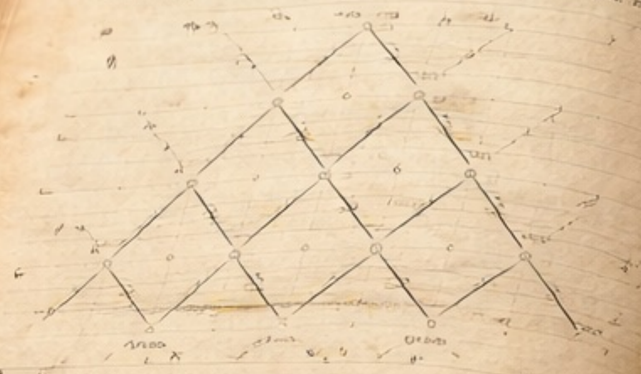


- A concrete domain C
- An abstract domain A
- A concretization function γ that relates the abstract domain to the concrete one
- An abstraction function α that connects the concrete domain to the abstract one
- Together they form a Galois Connection
- In this lecture we first revisit elements of order theory (C and A are ordered) and formalize the concept of Galois Connection and Galois Insertion

Chapter 2:
Elements of Abstract Interpretation

Order theory:
a quick recap and something new

Domain Theory



Read Again!

Posets

Definition 2.1 (Partial order, Poset)**.**
A partial order $\sqsubseteq$ *on a set* X *is a relation* $\sqsubseteq \in X \times X$ *that is:*

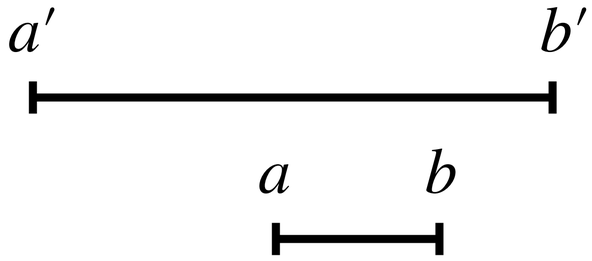
1. *reflexive:* $\forall x \in X : x \sqsubseteq x$;
2. *anti-symmetric:* $\forall x, y \in X : (x \sqsubseteq y) \land (y \sqsubseteq x) \implies x = y$;
3. *and transitive:* $\forall x, y, z \in X : (x \sqsubseteq y) \land (y \sqsubseteq z) \implies x \sqsubseteq z$.

Poset: examples

1. *Any* ***totally*** *ordered set is also a poset; for instance:* $(\mathbb{Z}, \leq)$.

2. *The parts of any set* X *ordered by inclusion,* $(\mathcal{P}(X), \subseteq)$, *is a poset. The order is not* ***total*** *; consider for instance that neither* $\{1\} \subseteq \{2\}$ *nor* $\{2\} \subseteq \{1\}$ *holds.*

3. $(\mathbb{Z}^2, \sqsubseteq)$, *the set of pairs of integers ordered as:* $(a, b) \sqsubseteq (a', b') \iff a \geq a' \wedge b \leq b'$ *is a poset. When interpreting the first element of each pair as the lower bound of an interval and the second element as its upper bound, this order is equivalent to interval inclusion.* ♦



Approximation

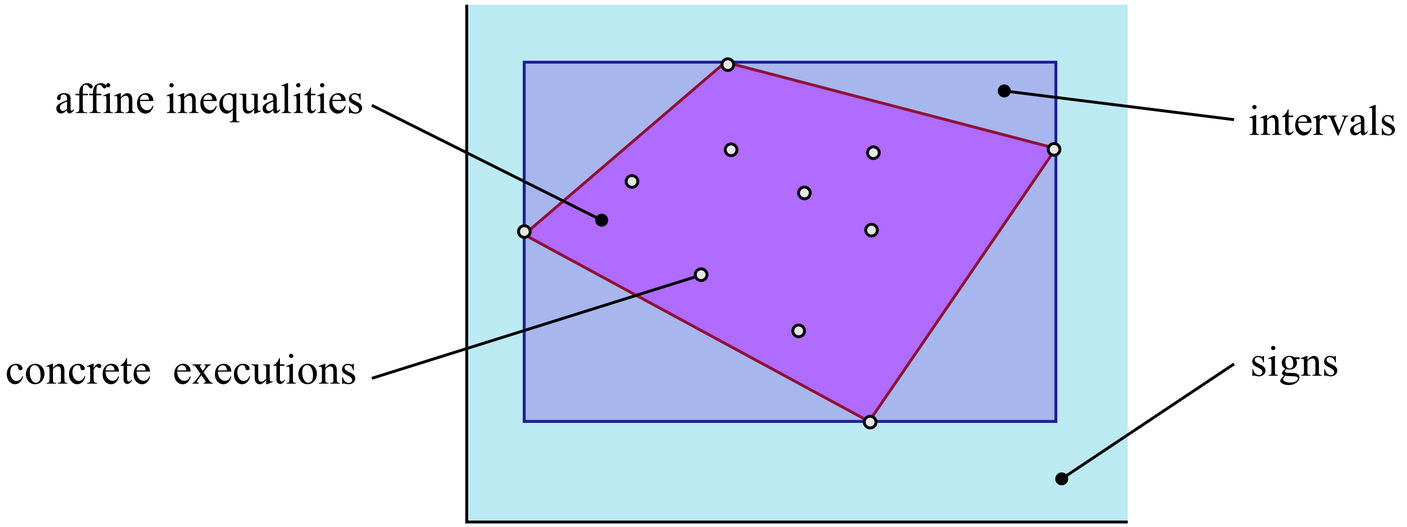
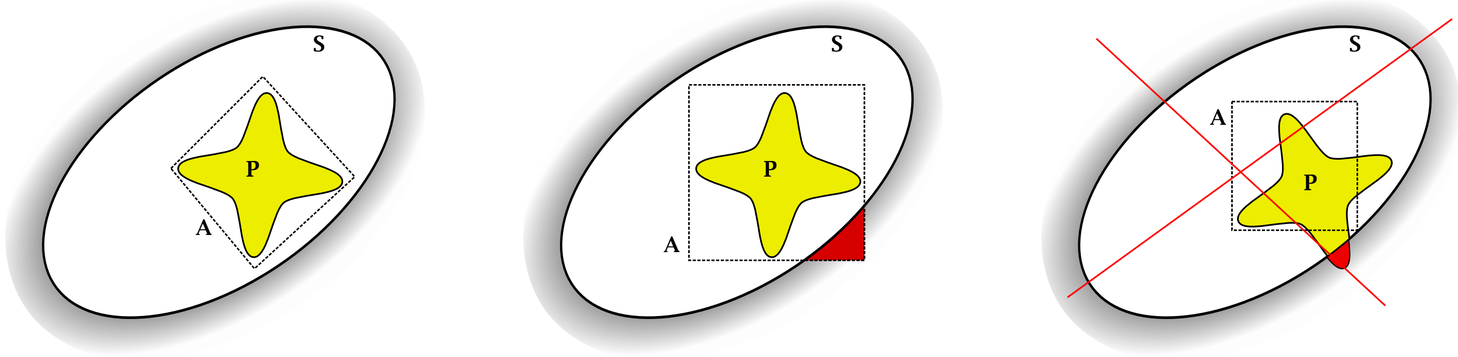


Figure 1.5: A set of points abstracted using affine inequalities (dark polyhedron), intervals (lighter rectangle) and signs (light quarter-plane).

1. Partial orders convey the idea of *approximation* (Fig. 1.5). Some analysis results may be coarser than some other results. The order is indeed partial as, sometimes, analysis results are incomparable. Consider, for instance, a variable that actually ranges in $[1, 9]$ and two sound interval analyses that output respectively $[0, 9]$ and $[1, 10]$.

Validation



precise analysis
$A \subseteq S \implies P \subseteq S$
(a)

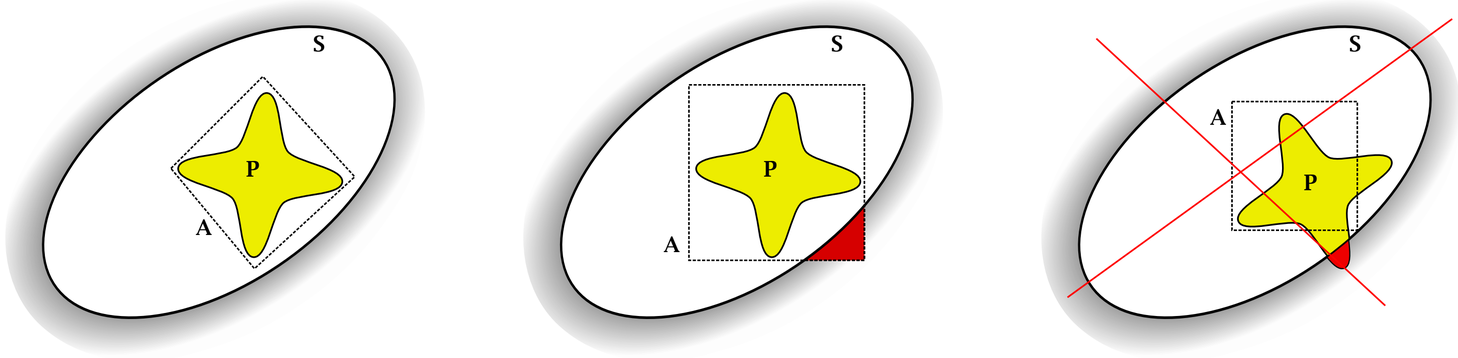
false alarm
$A \not\subseteq S$ but $P \subseteq S$
(b)

unsound analysis
$A \subseteq S$ but $P \not\subseteq S$
(c)

Figure 1.6: Proving that a program P satisfies a safety specification S, i.e., that $P \subseteq S$, using an abstraction A of P: (a) succeeds, (b) fails with a false alarm, and (c) is not a possible configuration for a sound analysis.

2. Partial orders convey the idea of *validating a specification* (Fig. 1.6). For instance, we can see a program P satisfying a specification S as the set inclusion $P \subseteq S$, meaning that all program behaviors are included in the authorized set of behaviors.

Soundness



precise analysis
$A \subseteq S \implies P \subseteq S$
(a)

false alarm
$A \not\subseteq S$ but $P \subseteq S$
(b)

unsound analysis
$A \subseteq S$ but $P \not\subseteq S$
(c)

Figure 1.6: Proving that a program P satisfies a safety specification S, i.e., that $P \subseteq S$, using an abstraction A of P: (a) succeeds, (b) fails with a false alarm, and (c) is not a possible configuration for a sound analysis.

3. Partial orders convey the idea of *soundness* (Fig. 1.6). An analysis is sound when it provably outputs a result that is coarser than the actual behavior.

Advancing towards invariant

```
    A = 0; B = 0;
1:  ⟅ A ∈ [0,0], B ∈ [0,0] ⟆
    while
2:      ⟅ A ∈ [0,100], B ∈ [0,+∞] ⟆
        (A < 100) {
3:      ⟅ A ∈ [0,99], B ∈ [0,+∞] ⟆
        A = A + 1;
4:      ⟅ A ∈ [1,100], B ∈ [0,+∞] ⟆
        B = B + 1;
5:      ⟅ A ∈ [1,100], B ∈ [1,+∞] ⟆
    }
6:  ⟅ A ∈ [100,100], B ∈ [0,+∞] ⟆
```

(a)

iteration	A	B
1	$[0,0]$	$[0,0]$
2	$[0,1]$	$[0,1]$
3	$[0,2]$	$[0,2]$
...	...	
100	$[0,99]$	$[0,99]$
101	$[0,100]$	$[0,100]$
102	$[0,100]$	$[0,101]$
103	$[0,100]$	$[0,102]$

(b)

Figure 1.4: Interval analysis of a simple loop (a) and the detailed iteration for location 2 (b).

4. Partial orders convey the idea of *iteration* (Fig. 1.4.(b)). When analyzing loops, a sequence of increasing semantic elements is computed, and the fact that they are ordered will be important to ensure that the computation is advancing towards a well-defined loop invariant.

$\sqcup$ is lub (join)

Given two elements a and b in a poset X, we call *upper bound* of a and b any element $c \in X$ such that $a \sqsubseteq c$ and $b \sqsubseteq c$, i.e., c is greater than both a and b. Likewise, a *lower bound* c of a and b satisfies $c \sqsubseteq a$ and $c \sqsubseteq b$. Moreover, c is the *least upper bound*, also called *lub* or *join* if it is the smallest element greater than both a and b. Such a least upper bound does not necessarily exist but, when it does, it is unique. It is written as $a \sqcup b$. Likewise, the unique *greatest lower bound* of a and b, also called *glb* or *meet*, if it exists, is the greatest element smaller than a and b, and it is denoted as $a \sqcap b$.

As $\sqcup$ and $\sqcap$ are associative operators, we will employ also the notations $\sqcup A$ and $\sqcap A$ to compute lubs an glbs on arbitrary (possibly infinite) sets A of elements. Finally, we denote respectively as $\bot$ (called *bottom*) and $\top$ (called *top*) the *least element* and the *greatest element* in the poset, if they exist.

$\sqcap$ is glb (meet)

Given two elements a and b in a poset X, we call *upper bound* of a and b any element $c \in X$ such that $a \sqsubseteq c$ and $b \sqsubseteq c$, i.e., c is greater than both a and b. Likewise, a *lower bound* c of a and b satisfies $c \sqsubseteq a$ and $c \sqsubseteq b$. Moreover, c is the *least upper bound*, also called *lub* or *join*, if it is the smallest element greater than both a and b. Such a least upper bound does not necessarily exist but, when it does, it is unique. It is written as $a \sqcup b$. Likewise, the unique *greatest lower bound* of a and b, also called *glb* or *meet*, if it exists, is the greatest element smaller than a and b, and it is denoted as $a \sqcap b$.

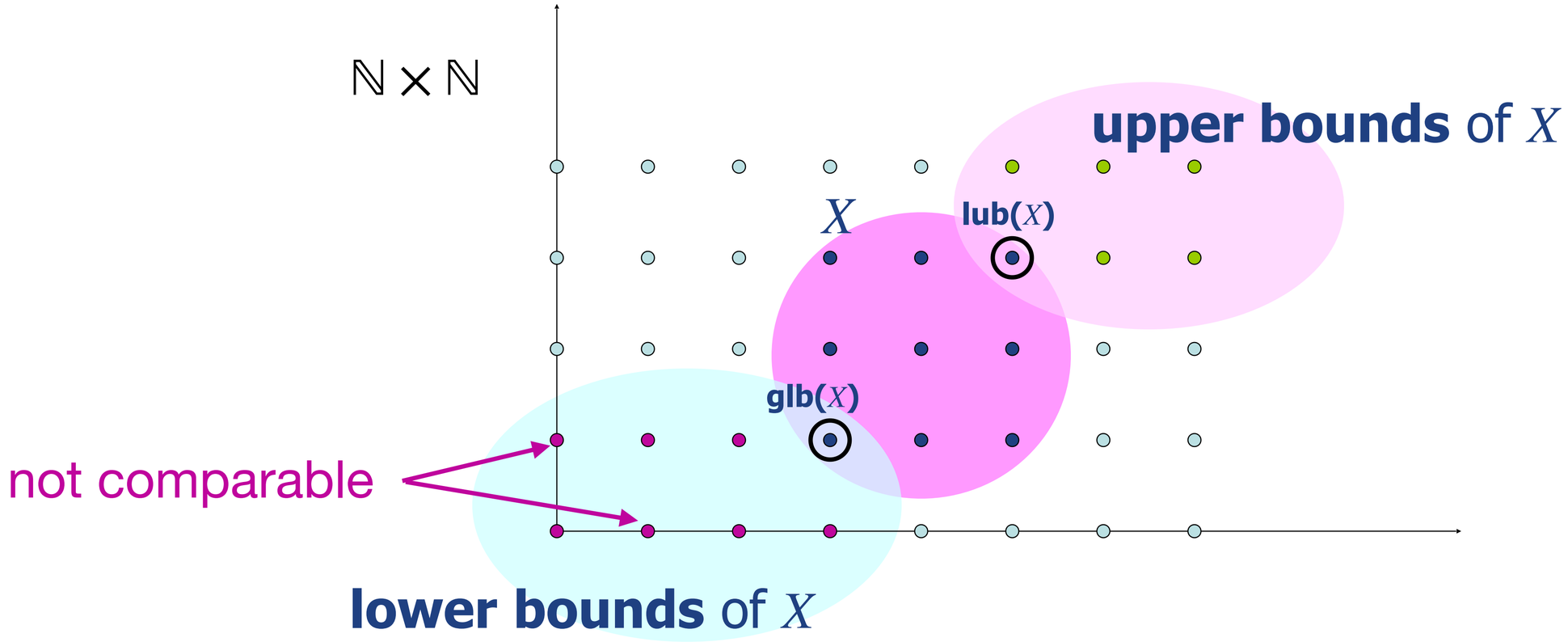
As $\sqcup$ and $\sqcap$ are associative operators, we will employ also the notations $\bigsqcup A$ and $\bigsqcap A$ to compute lubs an glbs on arbitrary (possibly infinite) sets A of elements. Finally, we denote respectively as $\bot$ (called *bottom*) and $\top$ (called *top*) the *least element* and the *greatest element* in the poset, if they exist.

Arbitrary lub / glb, bottom, top

Given two elements a and b in a poset X, we call *upper bound* of a and b any element $c \in X$ such that $a \sqsubseteq c$ and $b \sqsubseteq c$, i.e., c is greater than both a and b. Likewise, a *lower bound* c of a and b satisfies $c \sqsubseteq a$ and $c \sqsubseteq b$. Moreover, c is the *least upper bound*, also called *lub* or *join*, if it is the smallest element greater than both a and b. Such a least upper bound does not necessarily exist but, when it does, it is unique. It is written as $a \sqcup b$. Likewise, the unique *greatest lower bound* of a and b, also called *glb* or *meet*, if it exists, is the greatest element smaller than a and b, and it is denoted as $a \sqcap b$.

As $\sqcup$ and $\sqcap$ are associative operators, we will employ also the notations $\sqcup A$ and $\sqcap A$ to compute lubs an glbs on arbitrary (possibly infinite) sets A of elements. Finally, we denote respectively as $\bot$ (called *bottom*) and $\top$ (called *top*) the *least element* and the *greatest element* in the poset, if they exist

lub and glb



$$(x_1, y_1) \leq_{\mathbb{N}\times\mathbb{N}} (x_2, y_2) \iff x_1 \leq_{\mathbb{N}} x_2 \ \wedge \ y_1 \leq_{\mathbb{N}} y_2$$

pointwise ordering

lub and glb: examples

Example 2.2. *In the powerset poset $(\mathcal{P}(X), \subseteq)$ of any (possibly infinite) set X, the lub $\sqcup$ is simply the set union $\cup$, and the glb $\sqcap$ is simply the set intersection $\cap$. Both always exist for all arguments — hence, we actually have a complete lattice structure, as described in Sect. 2.1.5. We also have $\bot = \emptyset$ and $\top = X$.* ⧫

Example 2.3. *Not all posets have upper bounds or lubs. Consider, for instance, the poset $(\{a, b\}, =)$, where the partial order is the equality. Then, there is no upper bound for a and b, and so, there is no lub. Likewise, we can construct posets where a pair of elements have several upper bounds but no least upper bound, and posets where finite sets of elements have lubs but infinite sets of elements do not.* ⧫

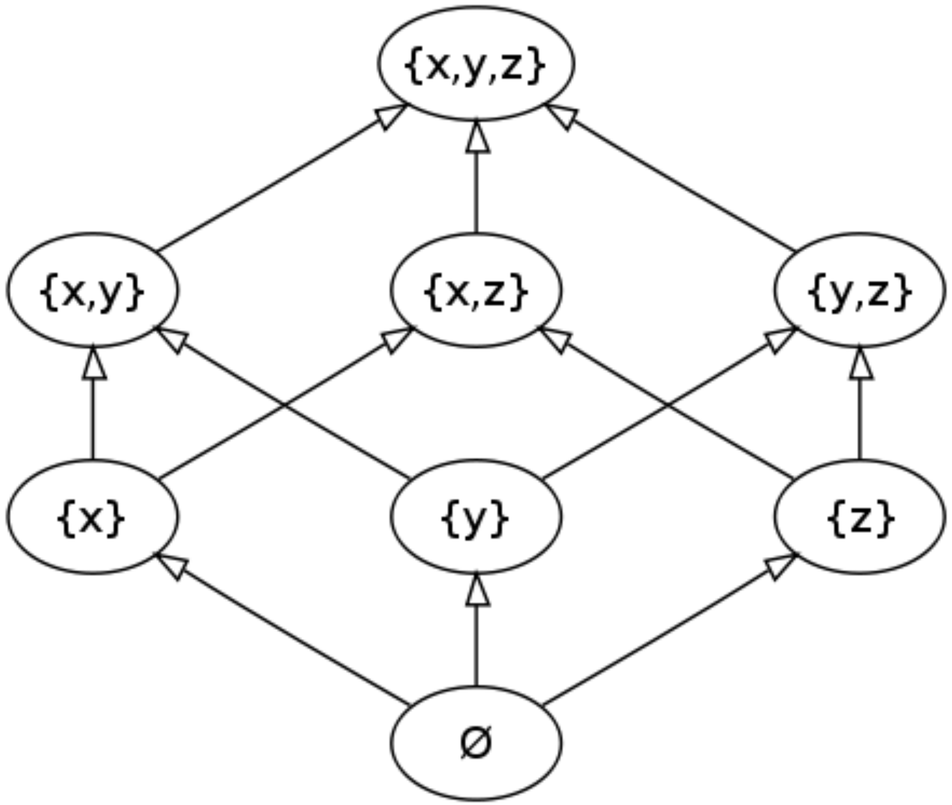
Example

$(\wp(\{x, y, z\}), \sqsubseteq)$

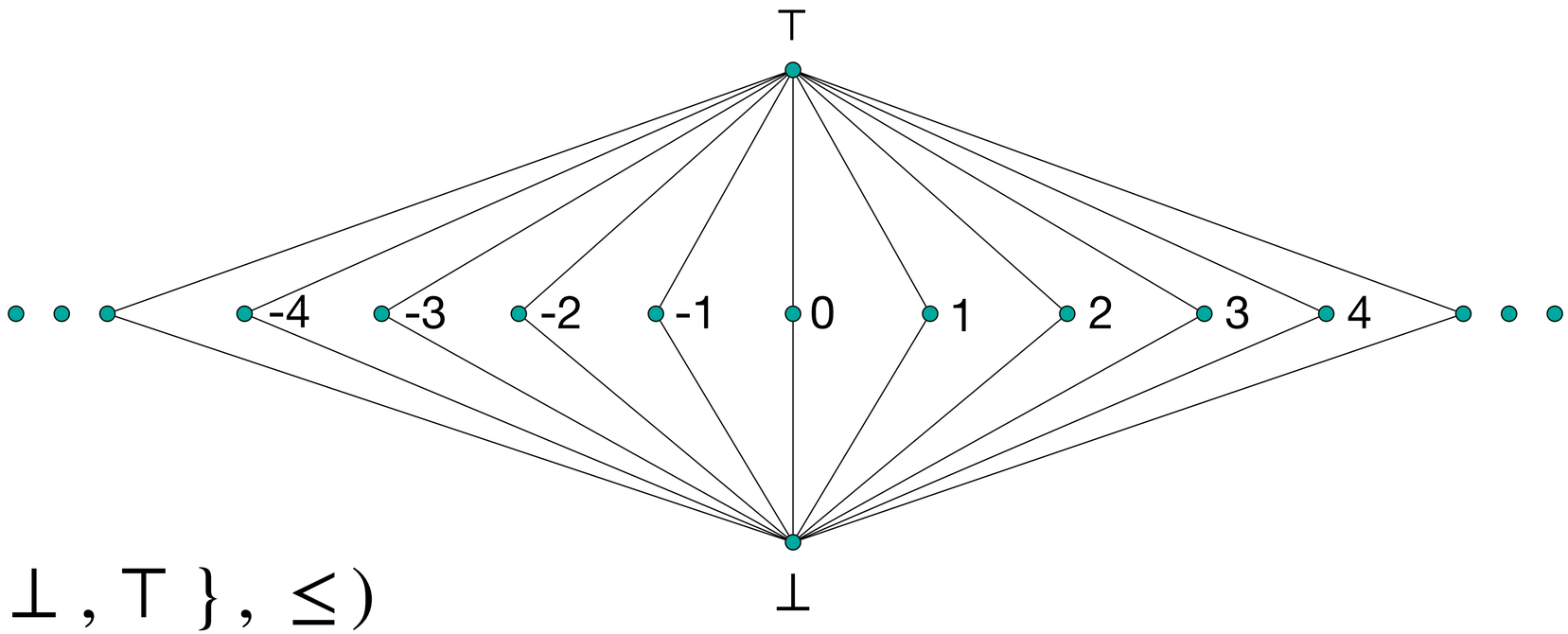
$Y \subseteq \wp(\{x, y, z\})$

$\text{lub}(Y) = \bigcup Y$

$\text{glb}(Y) = \bigcap Y$



Example



$(\mathbb{Z} \cup \{ \bot, \top \}, \leq)$

$n \in \mathbb{Z} \Rightarrow \bot \leq n \leq \top$

min and max vs. glb and lub

Remark 2.1. *We will also use the notation* $\min A$ *and* $\max A$ *to denote, respectively, the minimum and the maximum element in a set* $A \subseteq X$. *Note the difference between* $\min A$ *(resp.* $\max A$*) and* $\sqcap A$ *(resp.* $\sqcup A$*): the former imposes that the element smaller (resp. larger) than all the elements in* A *is in* A*, while the later may denote an element in* X *outside* A. *For instance, in the classic order* $(\mathbb{R}, \leq)$, $0 \sqcup 1 = \max\{0, 1\} = 1 \in \{0, 1\}$, *while* $\sqcup\{\, x \in \mathbb{R} \mid x < 1 \,\} = 1 \notin \{\, x \in \mathbb{R} \mid x < 1 \,\}$. *Naturally,* $\min A$ *and* $\max A$ *are not always defined for every set* A *in every poset, even less so than* $\sqcap A$ *and* $\sqcup A$. $\Diamond$

Chains (again, but not the same)

Definition 2.2 (Chain). *A* chain $C \subseteq X$ *in a poset* $(X, \sqsubseteq)$ *is a subset* C *of* X *that is totally ordered:* $\forall c, d \in C : (c \sqsubseteq d) \vee (d \sqsubseteq c)$. ■

A poset $(X, \sqsubseteq)$ *does not have infinite chains* if every chain is finite

A poset $(X, \sqsubseteq)$ has *finite height* $n \in \mathbb{N}$ if the cardinality of the longest chain is $n + 1$

A denumerable sequence $\{x_i\}_{i \in \mathbb{N}}$ of elements in $(X, \sqsubseteq)$ is an ascending chain if

$$n \leq m \Rightarrow x_n \sqsubseteq x_m$$

Convergence and ACC

A denumerable sequence $\{x_i\}_{i \in \mathbb{N}}$ (finitely) converges if

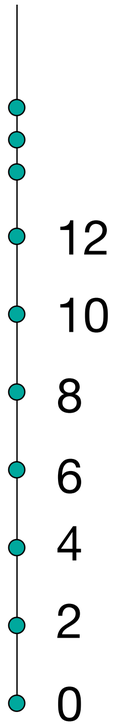
$$\exists n_0.\ \forall n \geq n_0.\ \ x_n = x_{n_0}$$

A poset $(X, \sqsubseteq)$ satisfies the Ascending Chain Condition (ACC) when each ascending chain converges (i.e., there are no infinite ascending chains)

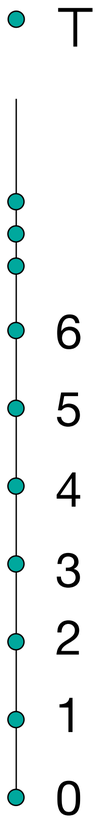
ACC abstract domains guarantee that the analysis always terminates!

Example

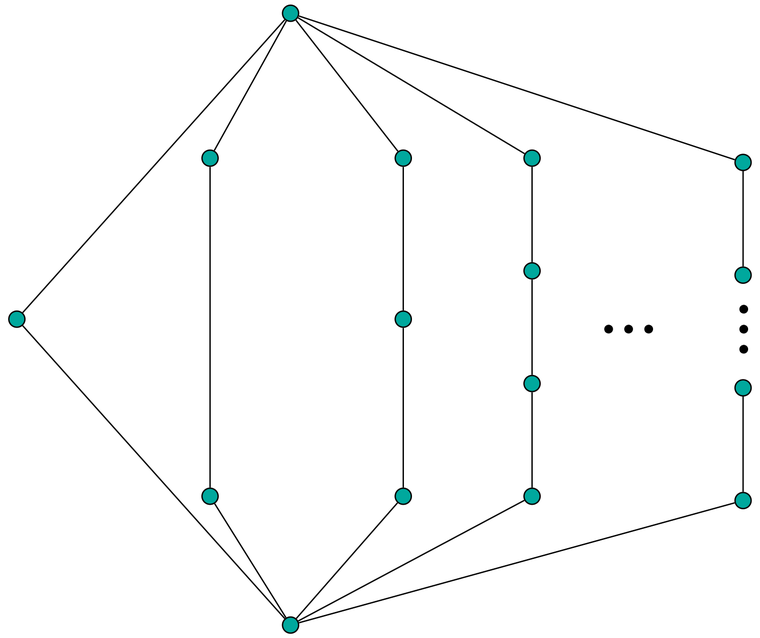
The total poset of even numbers is not ACC



This CPO is not ACC



This infinite complete lattice is ACC but does not have finite height



CPOs (again, but not the same)

Definition 2.3 (CPO)**.** *A* complete partial order*, or* CPO*, is a poset such that every chain has a lub.* ■

Note that $\emptyset$ is a chain, so, our definition requires that $\sqcup\emptyset$ exists. Stretching our definition a little, we state that $\sqcup\emptyset = \bot$ Indeed, the more elements we join, the larger the result is, so, when joining no element at all, we obtain the least element of all. As a consequence, all our CPO are so-called *pointed*, i.e., they feature a least element $\bot$.

Pointed CPOs are what we called CPOs with bottom

Example

In Fig. 2.1.(a), $\{a, c, f, g\}$ forms a chain.

Figure 2.1.(a) is a CPO. In fact, every finite poset is a CPO. Indeed every finite chain always has a lub.

Figure 2.1.(b) is not a CPO because infinite subsets of $\mathbb{N}$ do not have a lub in $\mathbb{N}$.

Figure 2.1.(c) is a CPO as every infinite subset of $\mathbb{N} \cup \{\infty\}$ has a lub: ∞.

For any (even infinite) set X, its powerset poset $(\mathcal{P}(X), \subseteq)$ is a CPO.

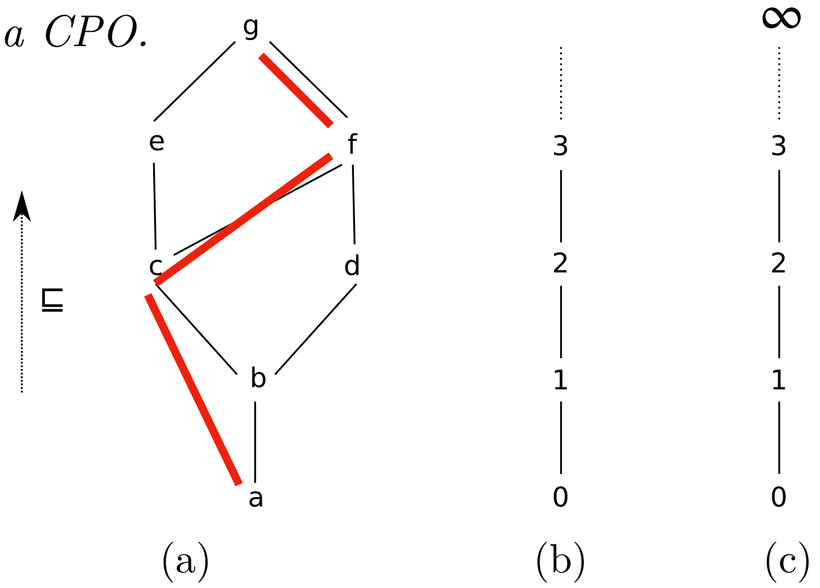


Figure 2.1: Hasse diagrams for: (a) a finite poset, (b) the natural integers $(\mathbb{N}, \leq)$, and (c) the natural integers enriched with infinity $(\mathbb{N} \cup \{\infty\}, \leq)$.

Complete lattices

Definition 2.4 (Lattice)**.** *A* lattice $(X, \sqsubseteq, \sqcup, \sqcap)$ *is a poset such that* $\forall a, b \in X : a \sqcup b$ *and* $a \sqcap b$ *exist.* ■

Definition 2.5 (Complete lattice)**.** *A* complete lattice $(X, \sqsubseteq, \sqcup, \sqcap, \bot, \top)$ *is a poset such that:*

1. $\forall A \subseteq X : \sqcup A$ *exists;* implies 3 and 4: $\bot = \text{lub}(\varnothing)$ and $\top = \text{lub}(X)$
2. $\forall A \subseteq X : \sqcap A$ *exists;* implies 3 and 4: $\bot = \text{glb}(X)$ and $\top = \text{glb}(\varnothing)$
3. X *has a least element* $\bot$*;* (unnecessary requirement)
4. X *has a greatest element* $\top$*.* (unnecessary requirement) ■

The lattice $\mathbb{N}$

$(\mathbb{N}, \leq)$

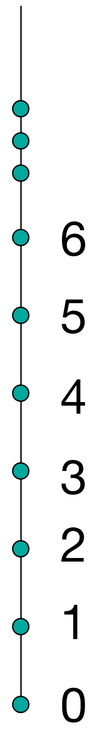
total order

lub = max

glb = min

it is a lattice, but not a complete lattice

it is not a CPO



The complete lattice $\mathbb{N}^{\top}$

$(\mathbb{N} \cup \{ \top \}, \leq)$

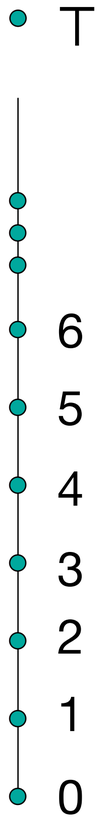
total order

lub = sup

glb = min

it is a complete lattice

it is a CPO



The complete lattice $\wp(X)$

1. *For any set X, the powerset:*

$$(\mathcal{P}(X), \subseteq, \cup, \cap, \emptyset, X) \tag{2.1}$$

is a complete lattice, as we can compute the intersection and union of arbitrary many (including infinitely many) sets. The Hasse diagram for $\mathcal{P}(\mathbb{Z})$ is presented in Fig. 2.2.

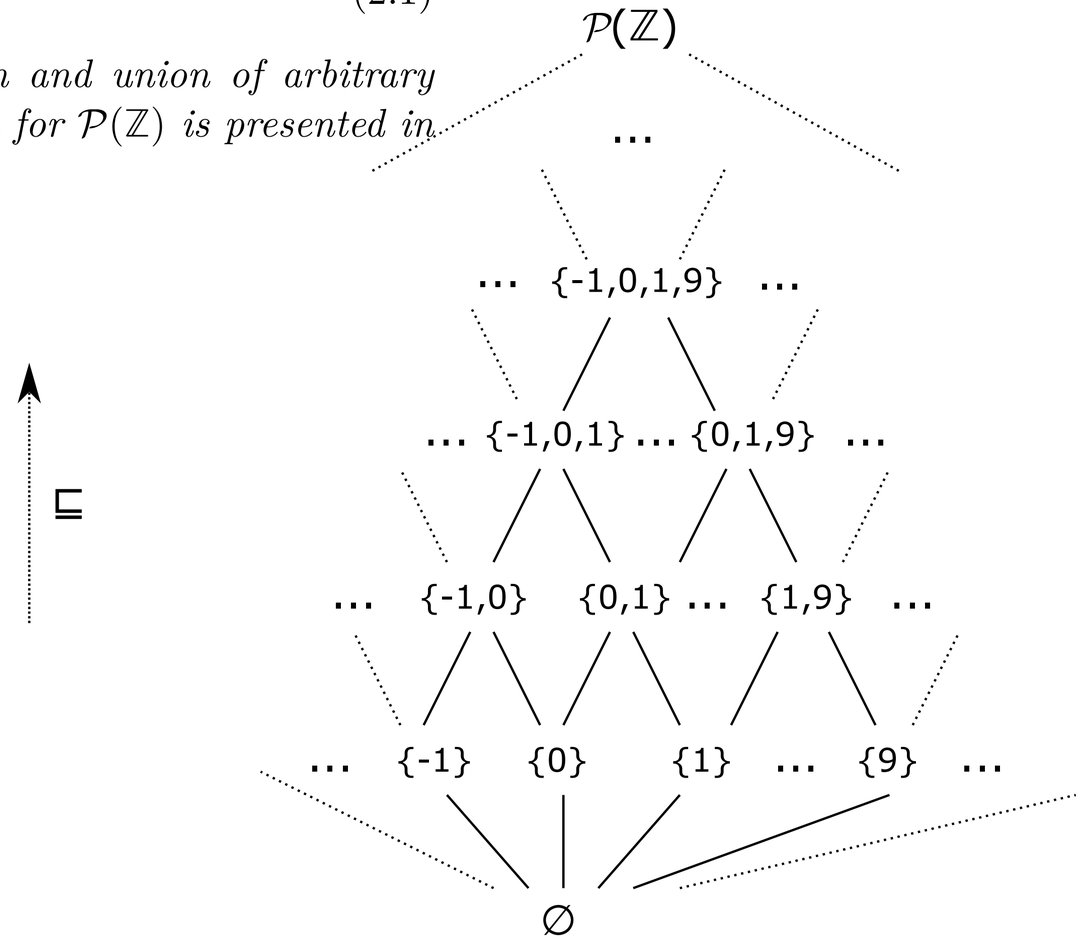


Figure 2.2: Hasse diagram for the powerset poset of integers $(\mathcal{P}(\mathbb{Z}), \subseteq)$.

The interval lattice

2. *We construct an integer interval lattice as follows:*

$$(\{\,[a,b] \mid a,b \in \mathbb{Z},\, a \leq b\,\} \cup \{\,\bot\,\}, \subseteq, \sqcup, \cap) \tag{2.2}$$

To simplify, we assimilate here a pair of bounds (a,b) *with the set* $[a,b]$ *of integers comprised between* a *and* b*, taking care that* $a \leq b$*. We add a special, unique, smallest element* $\bot$ *to represent* $\emptyset$*. We thus use the classic set operators:* $\subseteq$ *as partial order and* $\cap$ *as glb, as intervals are closed under intersection. However, they are not closed under set union, hence, we define the lub as* $[a,b] \sqcup [a',b'] \stackrel{\text{def}}{=} [\min(a,a'), \max(b,b')]$*, while* $\forall x\colon x \sqcup \bot = \bot \sqcup x = x$*. Indeed,* $[a,b] \sqcup [a',b']$ *computes the smallest interval containing intervals* $[a,b]$ *and* $[a',b']$*.*

Is it complete?  Not all lubs exist!

The interval complete lattice

3. *The integer interval lattice (2.2) from the previous point is not complete as the infinite family of intervals* $\{\,[0,i] \mid i \geq 0\,\}$ *has no lub. We complete the interval lattice into a complete lattice by allowing positive and negative infinities as bounds:*

$$(\{\,[a,b] \mid a \in \mathbb{Z} \cup \{\,-\infty\,\},\, b \in \mathbb{Z} \cup \{\,+\infty\,\},\, a \leq b\,\} \cup \{\,\bot\,\},\\ \subseteq,\, \sqcup,\, \cap,\, \bot,\, [-\infty,+\infty]) \tag{2.3}$$

Lubs are computed as: $\sqcup_{i\in I}\,[a_i,b_i] \stackrel{\text{def}}{=} [\min_{i\in I} a_i,\, \max_{i\in I} b_i]$. *Moreover, the lattice now has a greatest element:* $[-\infty,+\infty]$. *The complete lattice of intervals is illustrated in Fig. 2.3.*

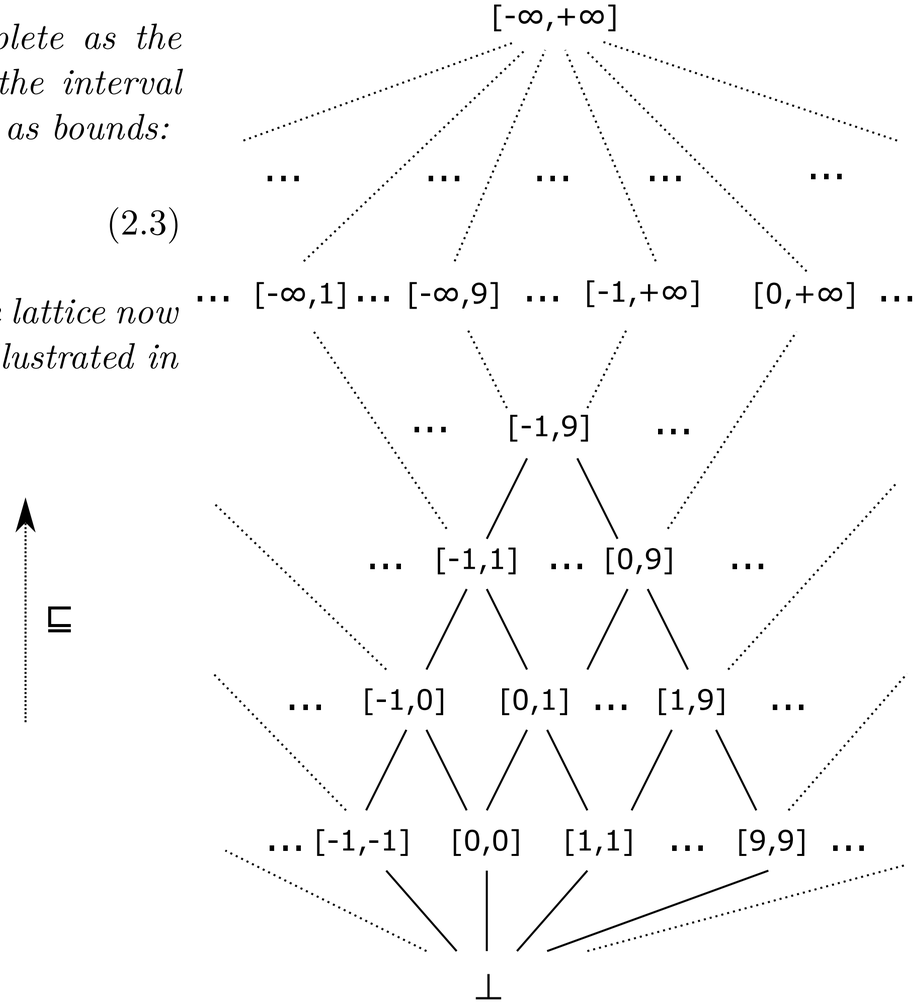


Figure 2.3: Hasse diagram for the interval lattice, with bounds extended to ∞.

Summary

A poset $(X, \sqsubseteq)$ is a *CPO* if every (possibly empty) chain has a lub (must have $\bot$)

A poset $(X, \sqsubseteq)$ is a *lattice* if every pair of elements has both lub and glb

A poset $(X, \sqsubseteq)$ is a *complete lattice* if every (possibly empty) subset has both lub and glb

$\bot = \text{lub}(\varnothing) = \text{glb}(X)$ and $\top = \text{lub}(X) = \text{glb}(\varnothing)$

lub and glb are commutative and associative

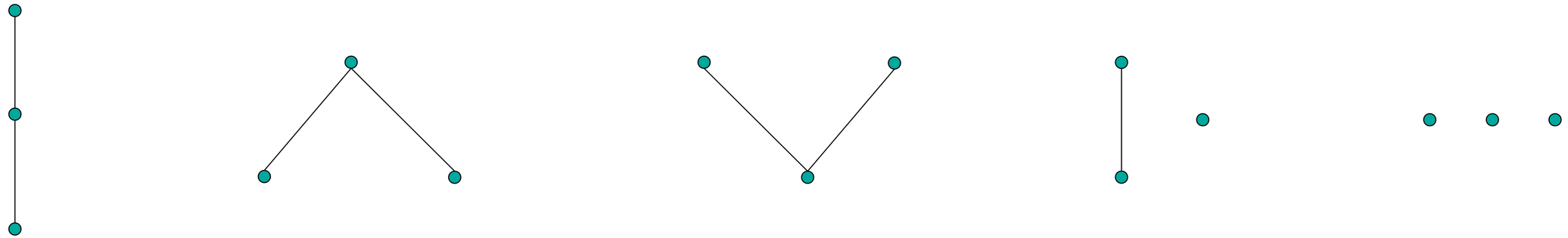
Each finite lattice is a complete lattice

Each complete lattice is a CPO (and a lattice)

Exercises

Exercise

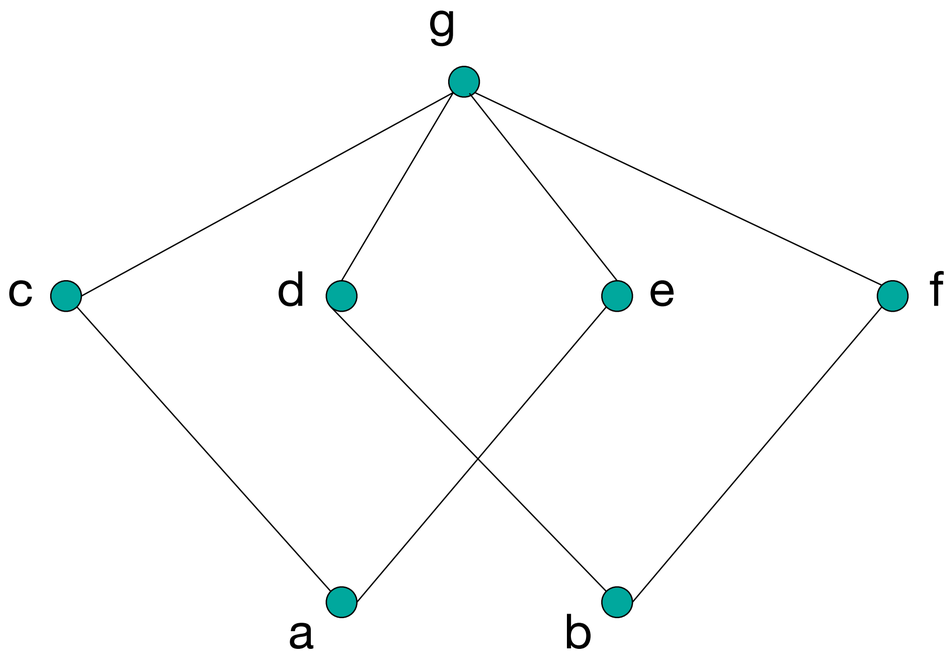
Classify these 3-elements posets



lattice?

complete
lattice?

Exercise



poset?

CPO?

lattice?

complete lattice?

Exercise

Definition 2.5 (Complete lattice)**.** *A* complete lattice $(X, \sqsubseteq, \sqcup, \sqcap, \bot, \top)$ *is a poset such that:*

1. $\forall A \subseteq X$: $\sqcup\ A$ *exists;*
2. $\forall A \subseteq X$: $\sqcap\ A$ *exists;*

prove that: 1 holds iff 2 holds

The divisibility lattice

4. *The* divisibility lattice *is defined as* $(\mathbb{N}^*, |, \mathrm{lcm}, \mathrm{gcd})$*, where* $x|y$ *means that* x *divides* y*, or equivalently that* y *is a multiple of* x*, i.e.,* $\exists k \in \mathbb{N}^* : xk = y$*. Then, the lub and glb are respectively the least common multiple,* lcm*, and the greatest common divisor,* gcd*. This lattice is illustrated in Fig. 2.4.*

Is it complete?

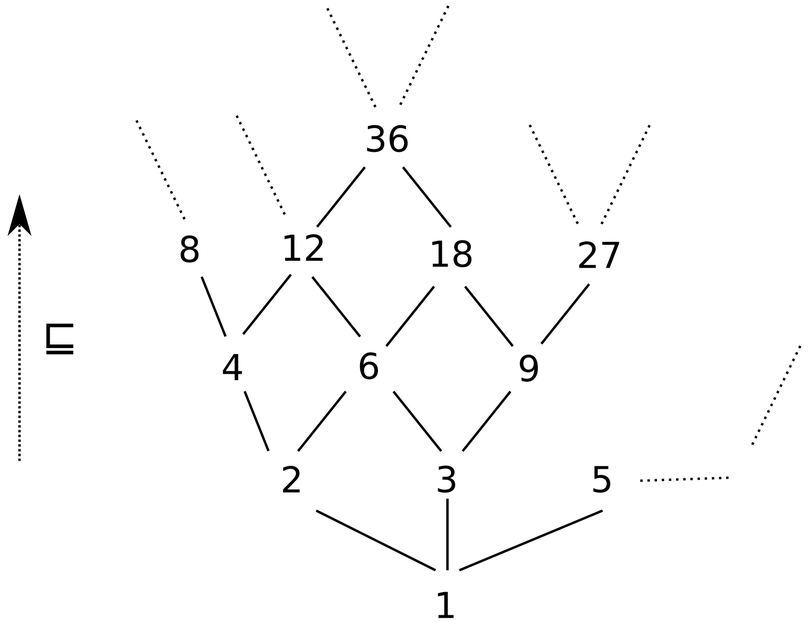


Figure 2.4: Hasse diagram for the divisor lattice $(\mathbb{N}^*, |, \mathrm{lcm}, \mathrm{gcd})$ where $x|y$ if x divides y

Abstraction and concretization

Monotonicity

Monotonicity: a function $f : (A_1, \sqsubseteq_1) \to (A_2, \sqsubseteq_2)$ *between two posets is* monotonic *if* $\forall x, y \in A_1 : x \sqsubseteq_1 y \implies f(x) \sqsubseteq_2 f(y)$.

These definitions can be justified informally. In terms of information theory, where the partial order measures a quantity of information, monotonicity means that, given more information as input, a function must output more information. In terms of program semantics, the more input we feed a program, the more behaviors it will exhibit.

Continuity

Continuity: a function $f : (A_1, \sqsubseteq_1, \sqcup_1) \to (A_2, \sqsubseteq_2, \sqcup_2)$ *between two CPO is* continuous *if for every chain* $C \subseteq A_1$, $\{ f(c) \mid c \in C \}$ *is also a chain and the limits coincide:* $f(\sqcup_1 C) = \sqcup_2 \{ f(c) \mid c \in C \}$.

Continuity implies monotonicity: when $x \sqsubseteq y$, simply consider the chain $\{x, y\}$ to get that $f(x) \sqcup f(y) = f(x \sqcup y) = f(y)$, i.e., $f(x) \sqsubseteq f(y)$. Continuity goes one step further and requires that functions preserve limits, i.e., there is "no surprise" at the limit and the result of $f(\sqcup A)$ can be intuited by observing the sequence $\{ f(a) \mid a \in A \}$. For instance, an operator f over $(\mathbb{N} \cup \{\infty\}, \leq)$ such that $f(x) = 0$ if $x \neq \infty$ but $f(\infty) = 1$ is not continuous, as $f(\infty)$ cannot be intuited from the set of all finite images $\{ f(x) \mid x \in \mathbb{N} \} = \{0\}$.

Extensivity and reductivity

Extensivity: an operator $f : (A, \sqsubseteq) \to (A, \sqsubseteq)$ *is* extensive *if* $\forall a \in A : a \sqsubseteq f(a)$, *and* reductive *if* $\forall a \in A : f(a) \sqsubseteq a$. ■

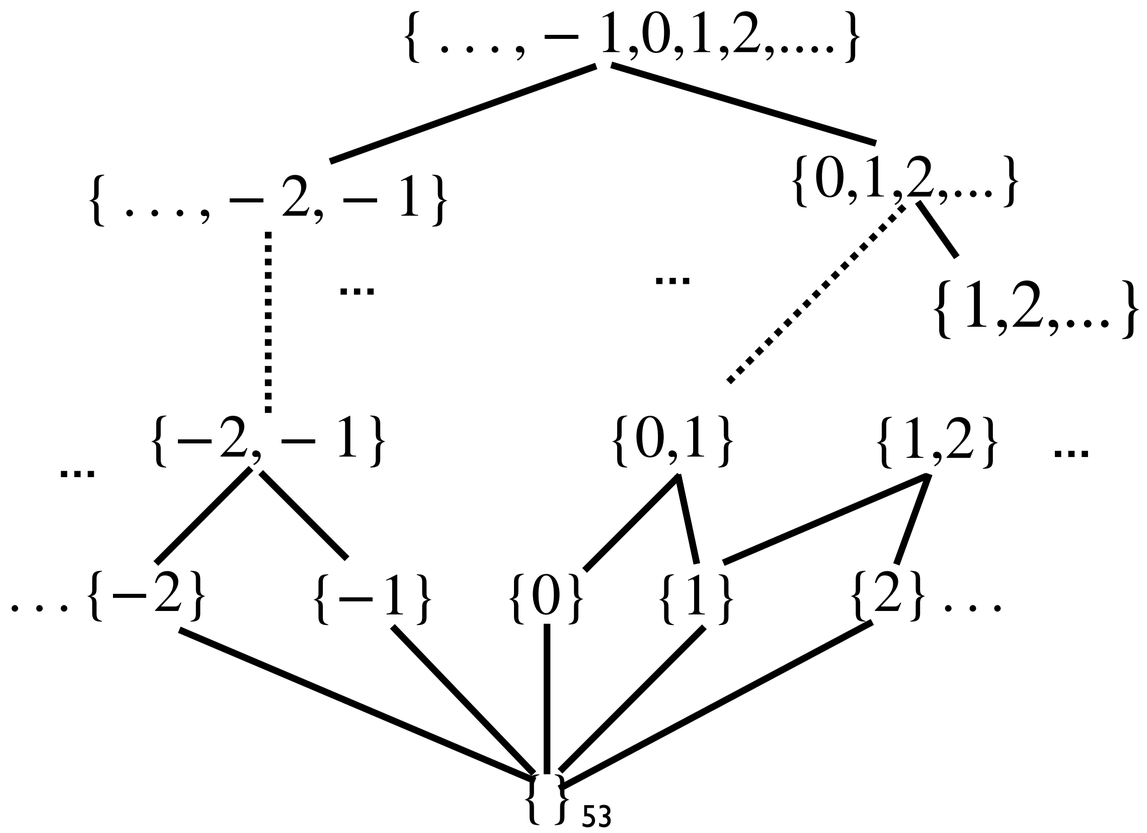
Extensivity is essentially useful when combined with monotonicity to bootstrap iteration: starting from an arbitrary x, we have $x \sqsubseteq f(x)$ by extensivity; then, applying monotonicity, $f(x) \sqsubseteq f^2(x)$, etc., so that the sequence $\{ f^i(x) \mid i \in \mathbb{N} \}$ is increasing.

In Abstract Interpretation, these properties often hold for concrete operators, but are often lost in the abstract world for the sake of generality — the same way abstract worlds feature less algebraic properties than concrete ones, abstract operators feature less algebraic properties than concrete ones.

Concrete domain

The set of values S that we would like to compute belongs to the concrete domain C

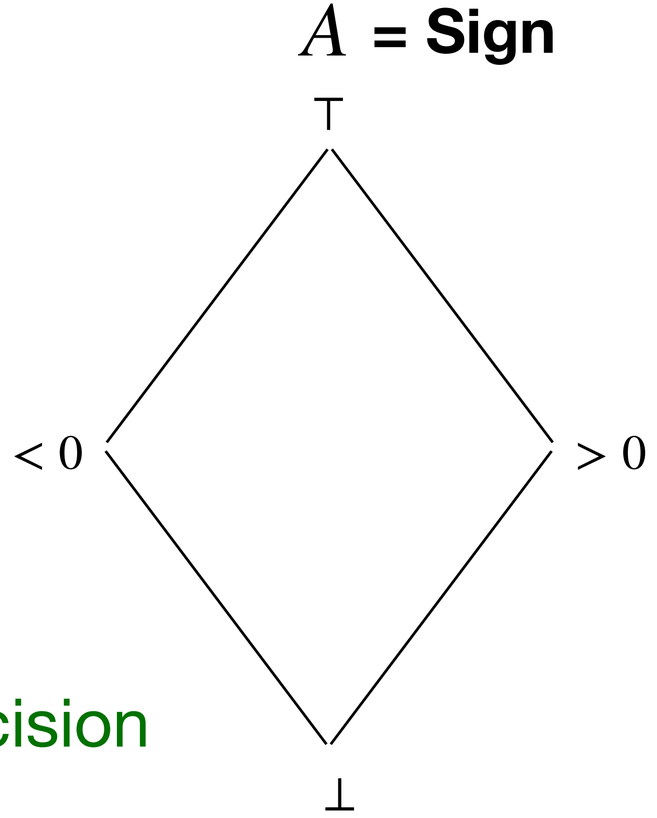
$(\wp(\mathbb{Z}), \subseteq)$



Abstract Domain

$(A, \sqsubseteq)$ expresses some properties of the concrete values

For example:
being positive or negative



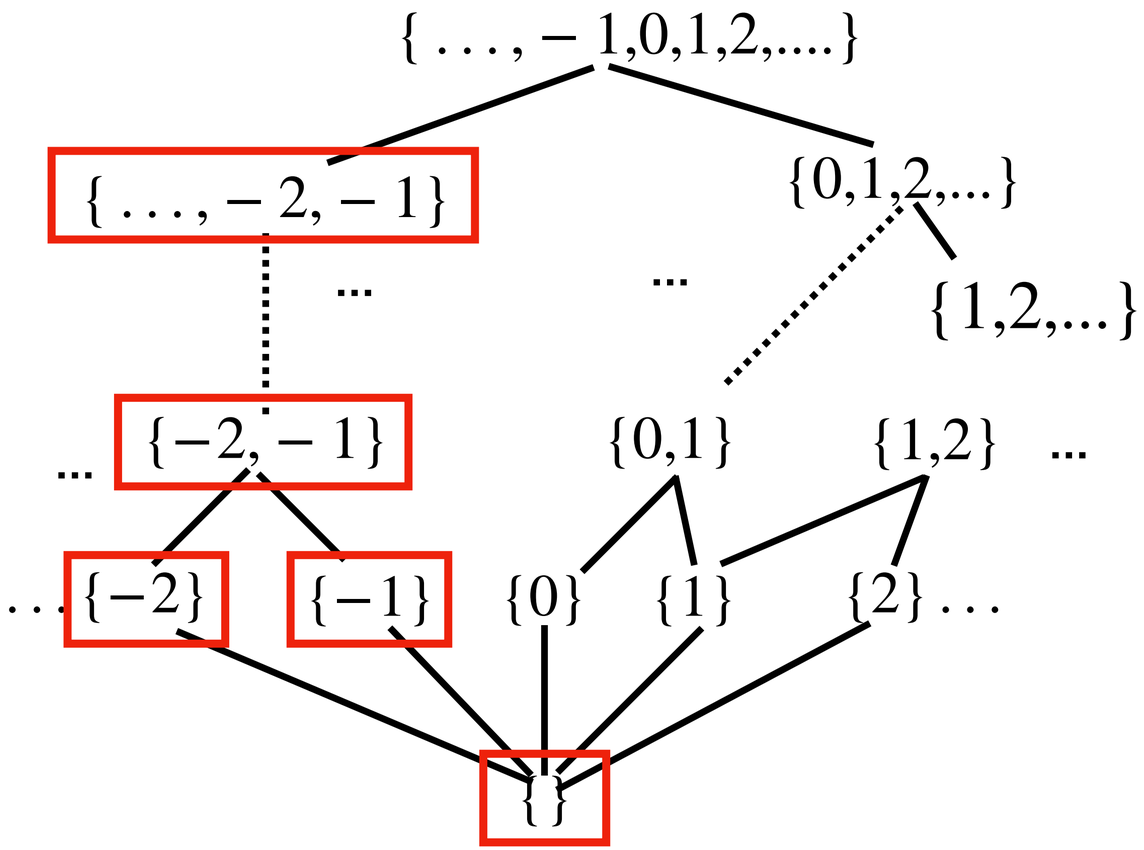
The order $\sqsubseteq$ on the abstract domain reflects the precision

Ingredients of Abstract Interpretation

- A concrete domain C
- An abstract domain A
- A concretization function $\gamma : A \to C$
  that relates the abstract domain to the concrete one
- An abstraction function $\alpha : C \to A$
  that connects the concrete domain to the abstract one

Sound approximation

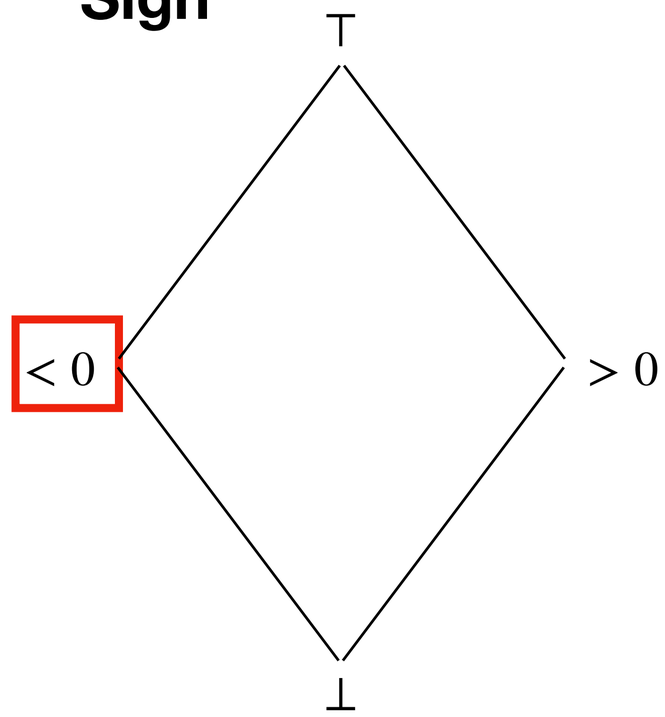
C



Any set that contains negative integers only

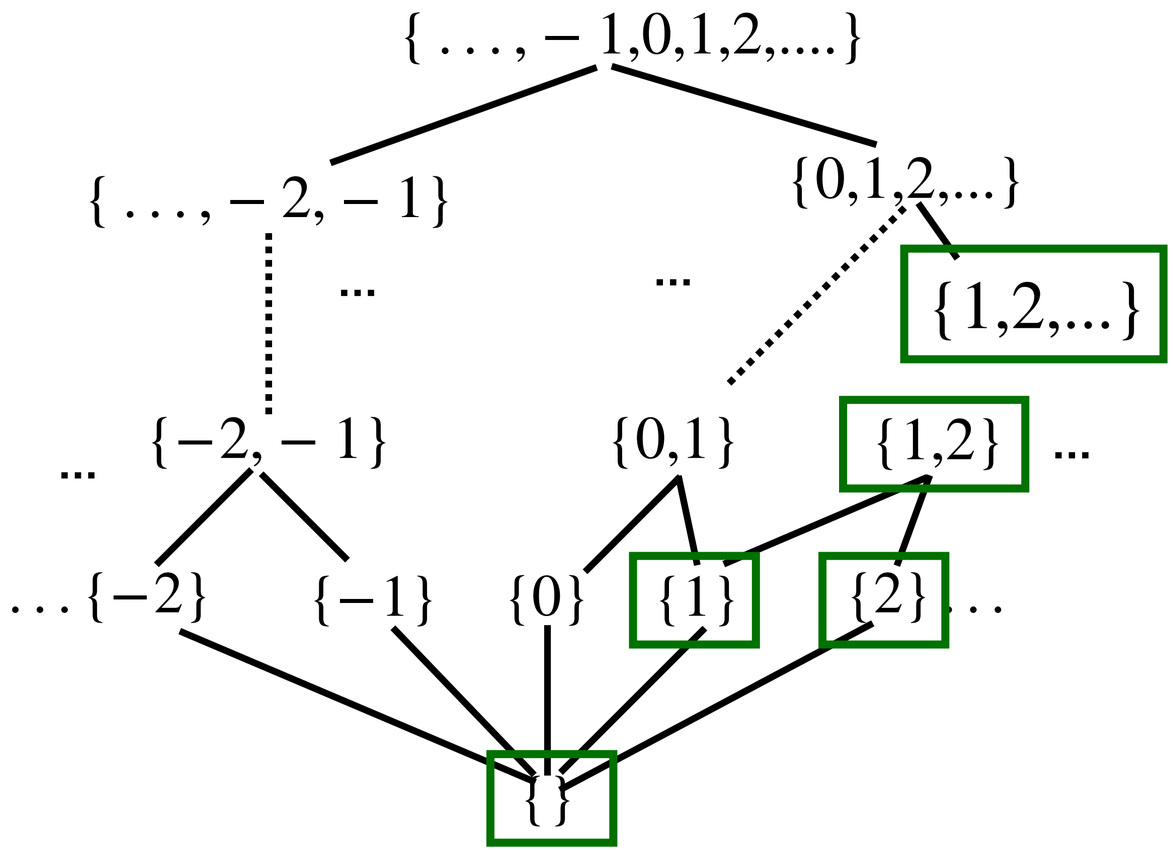
A

Sign



Sound approximation

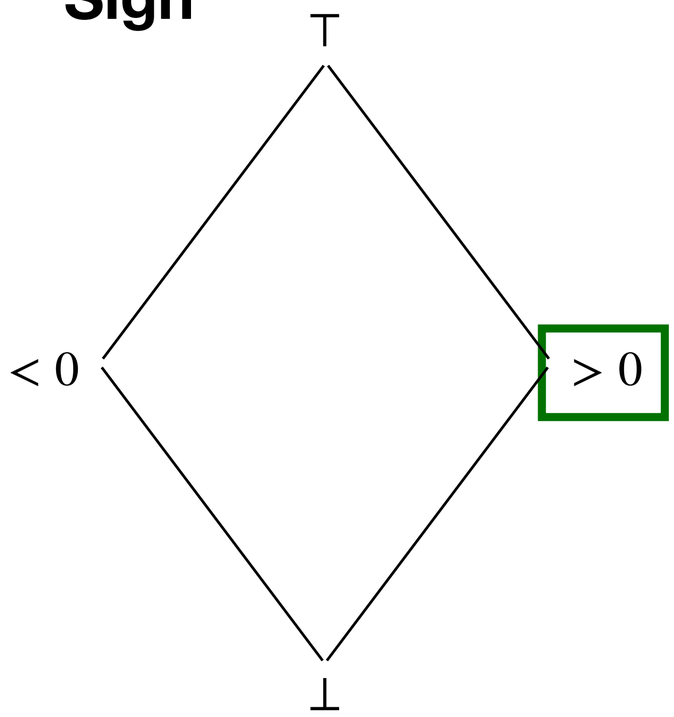
C



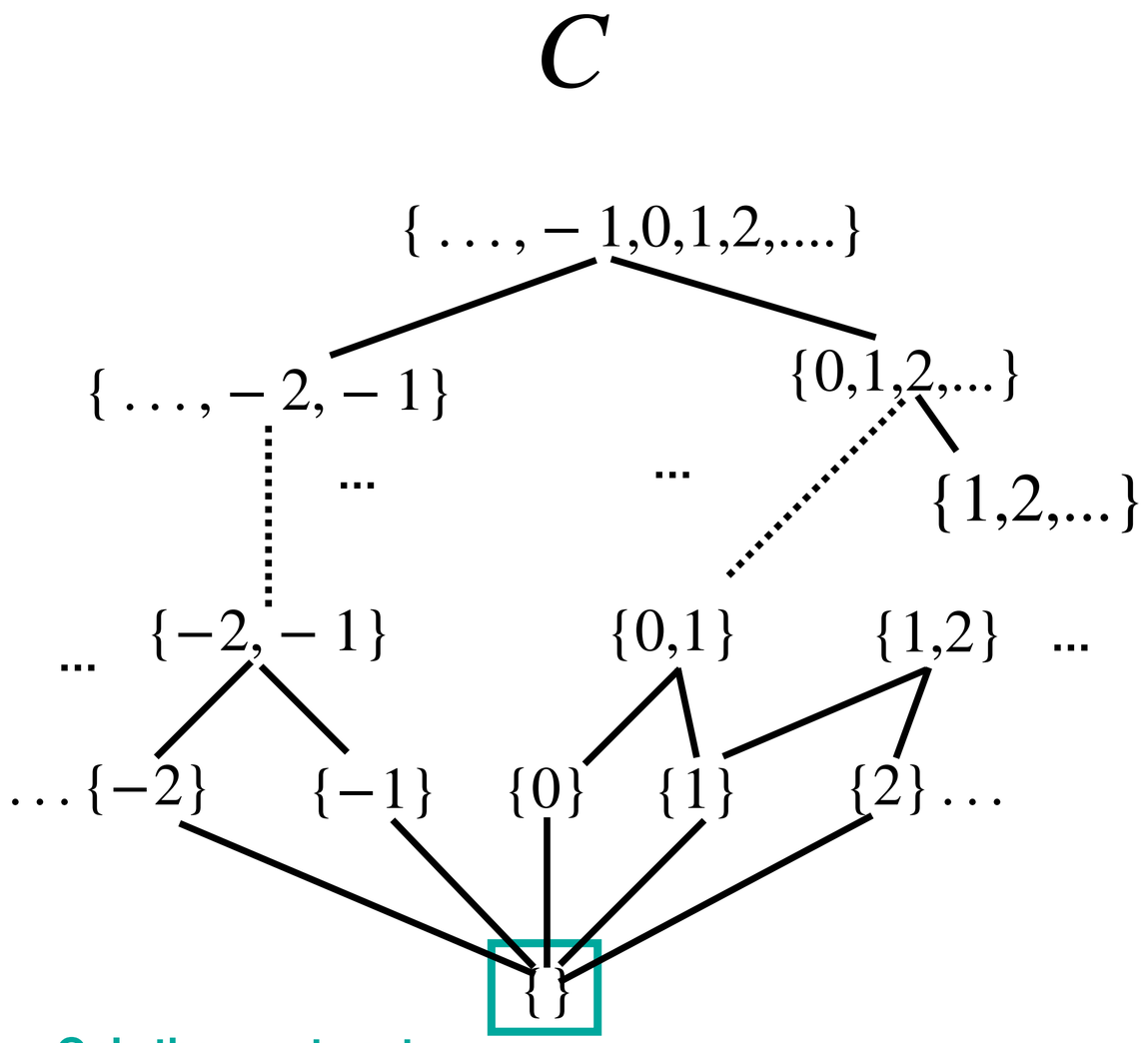
Any set that contains positive integers only

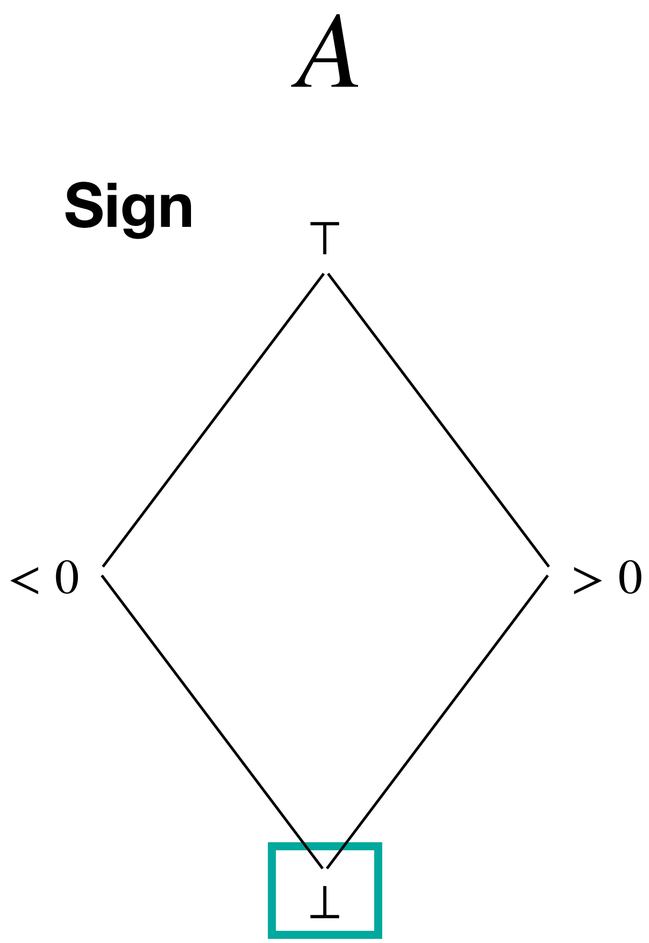
A

Sign



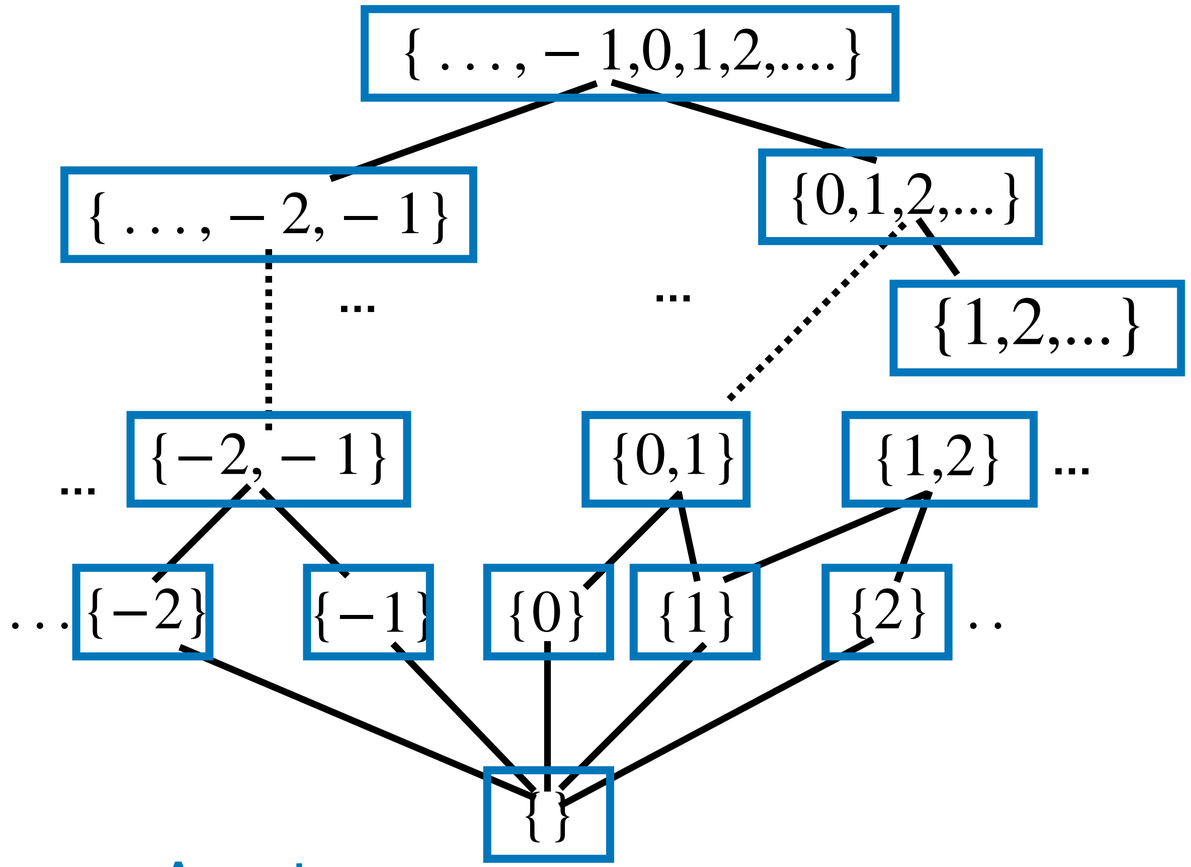
Sound approximation



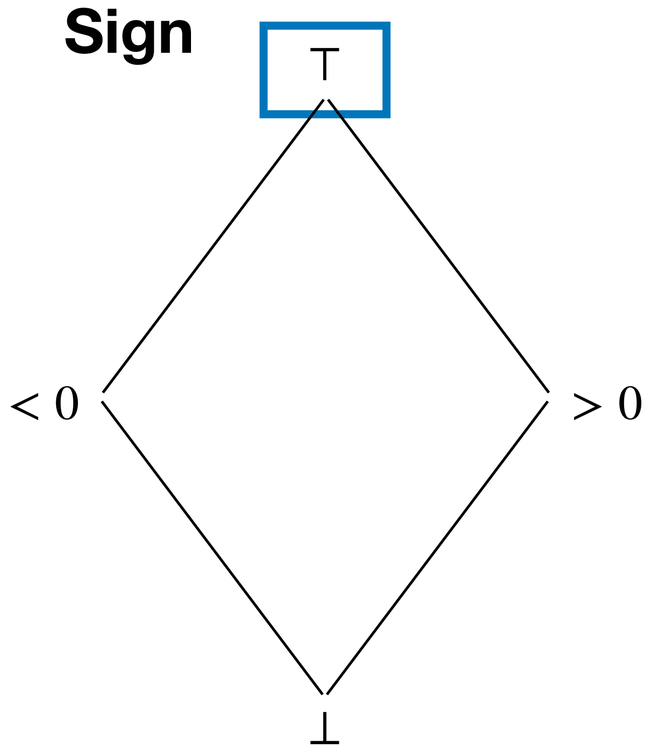


Sound approximation

C



A



Concretization

The minimum structure we require in the concrete and the abstract worlds is a partial order that models an amount of information — larger elements expose more program behaviors. Thus, the concrete world is a poset $(C, \leq)$, for instance integer powersets (2.1), and the abstract world is another poset $(A, \sqsubseteq)$, for instance intervals (2.3). The minimum connection we request between these worlds is a *concretization* function traditionally denoted as γ:

Definition 2.10 (Concretization)**.** *A concretization function* $\gamma \in (A, \sqsubseteq) \to (C, \leq)$ *is a* monotonic *function assigning a concrete meaning, in* C*, to each abstract element in* A*.* ■

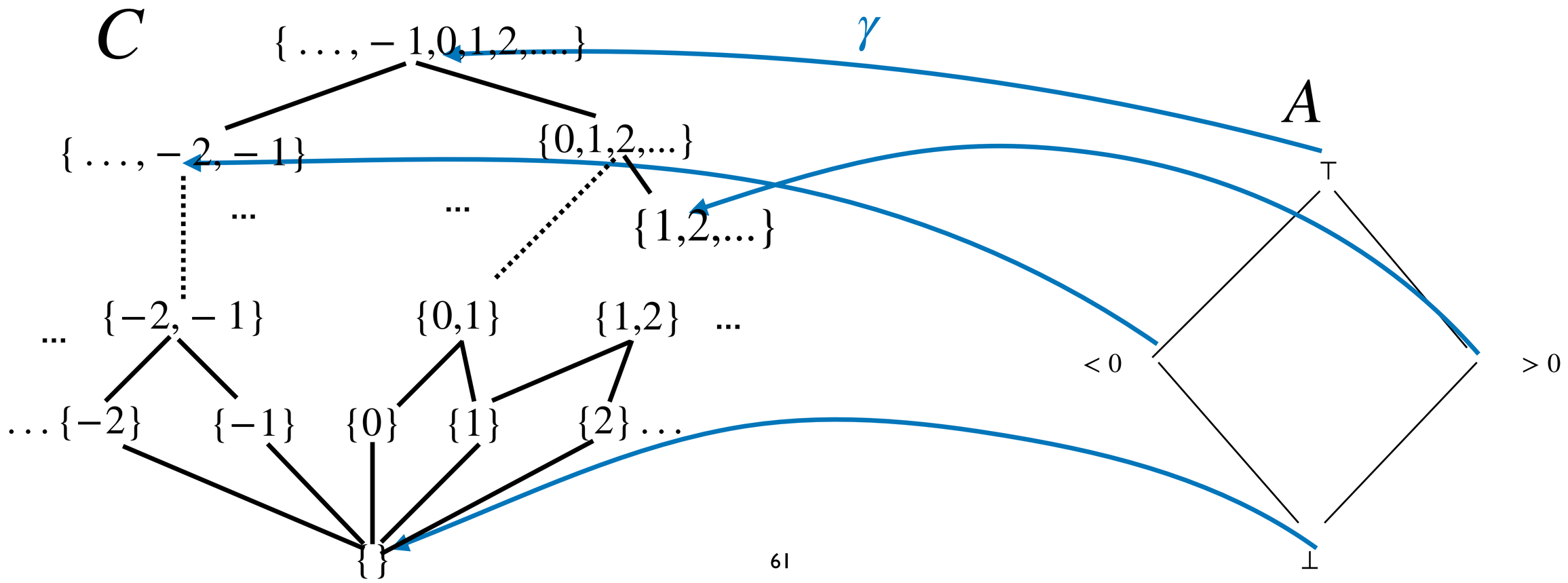
The monotonicity simply states that coarser abstract elements represent coarser concrete elements.

Concretization function

Definition

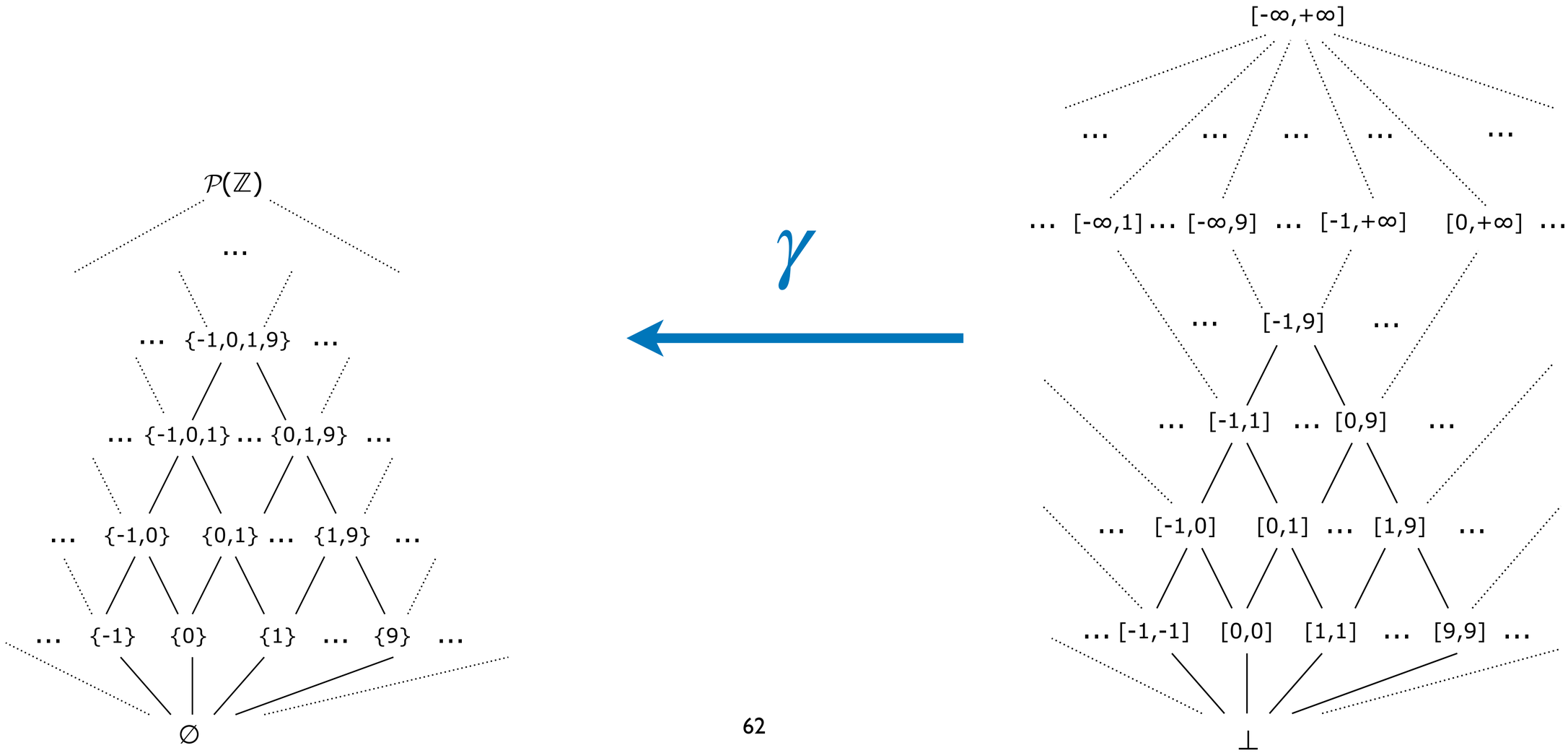
Concretization function $\gamma : A \rightarrow C$ is a monotone function

that maps abstract a into the greatest concrete c that it approximates



Example

The interval abstraction (2.3) already considers that an interval is a set of integers. The concretization from intervals to sets (2.1) is thus the identity.



Example

Consider an alternate abstract domain of intervals where an interval is represented by either a pair of bounds, *or* $\bot$*:*

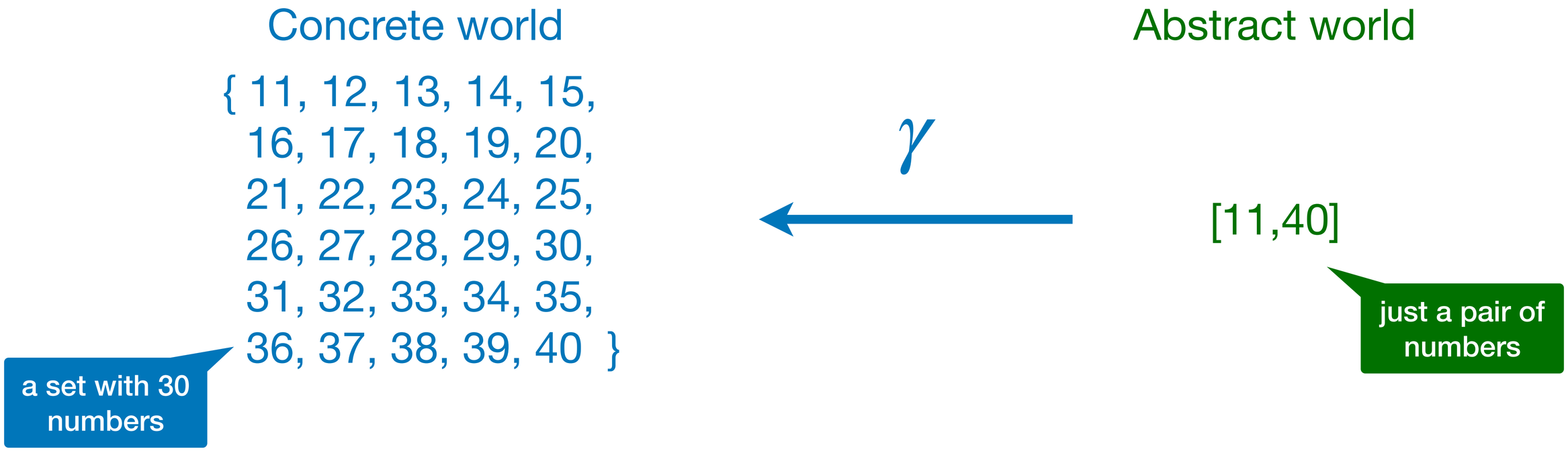
$$\{\,(a,b) \mid a \in \mathbb{Z} \cup \{\,-\infty\,\},\, b \in \mathbb{Z} \cup \{\,+\infty\,\},\, a \leq b\,\} \cup \{\,\bot\,\} \tag{2.4}$$

Then, the concretization is given by:

$$\begin{array}{lcl} \gamma((a,b)) & \stackrel{\text{def}}{=} & \{\,x \in \mathbb{Z} \mid a \leq x \leq b\,\} \\ \gamma(\bot) & \stackrel{\text{def}}{=} & \emptyset \end{array}$$

Note (Interval notation)**.** *Now that the distinction between the abstract world and the concrete world is clear, in the rest of the tutorial, when discussing intervals, we will write* $[a,b]$ *to actually denote the pair of bounds* (a,b) *that internally represents an interval, and write explicitly* $\gamma([a,b])$ *when discussing about the set of integers in this interval.* $\Diamond$

Example



Note (Interval notation). *Now that the distinction between the abstract world and the concrete world is clear, in the rest of the tutorial, when discussing intervals, we will write* $[a, b]$ *to actually denote the pair of bounds* (a, b) *that internally represents an interval, and write explicitly* $\gamma([a, b])$ *when discussing about the set of integers in this interval.* $\Diamond$

Multiple representatives?

Note that γ does not need to be onto — i.e., injective. It is possible to imagine an abstract world where several abstract elements represent the same concrete element. Consider, for instance, a variant of (2.4), $\{ (a, b) \mid a \in \mathbb{Z} \cup \{ -\infty \}, b \in \mathbb{Z} \cup \{ +\infty \} \}$, where we do not enforce $a \leq b$, and thus, any pair such that $a > b$ is a representation for the empty set. This is mainly useful, as we will see later, in the case of domains where computing unique, normal forms for an abstract element is not possible or would be too time-consuming.

Sound and exact abstractions

Definition 2.11 (Soundness, Exactness)**.**
$a \in A$ *is a* sound *abstraction of* $c \in C$ *if and only if* $c \leq \gamma(a)$.
It is moreover exact *if* $c = \gamma(a)$. (also called expressible) ■

The case where the equality holds corresponds to a rare case where the concrete element can be exactly be represented in the abstract (such as, for instance, a contiguous set of integers, which can be represented exactly as an interval).

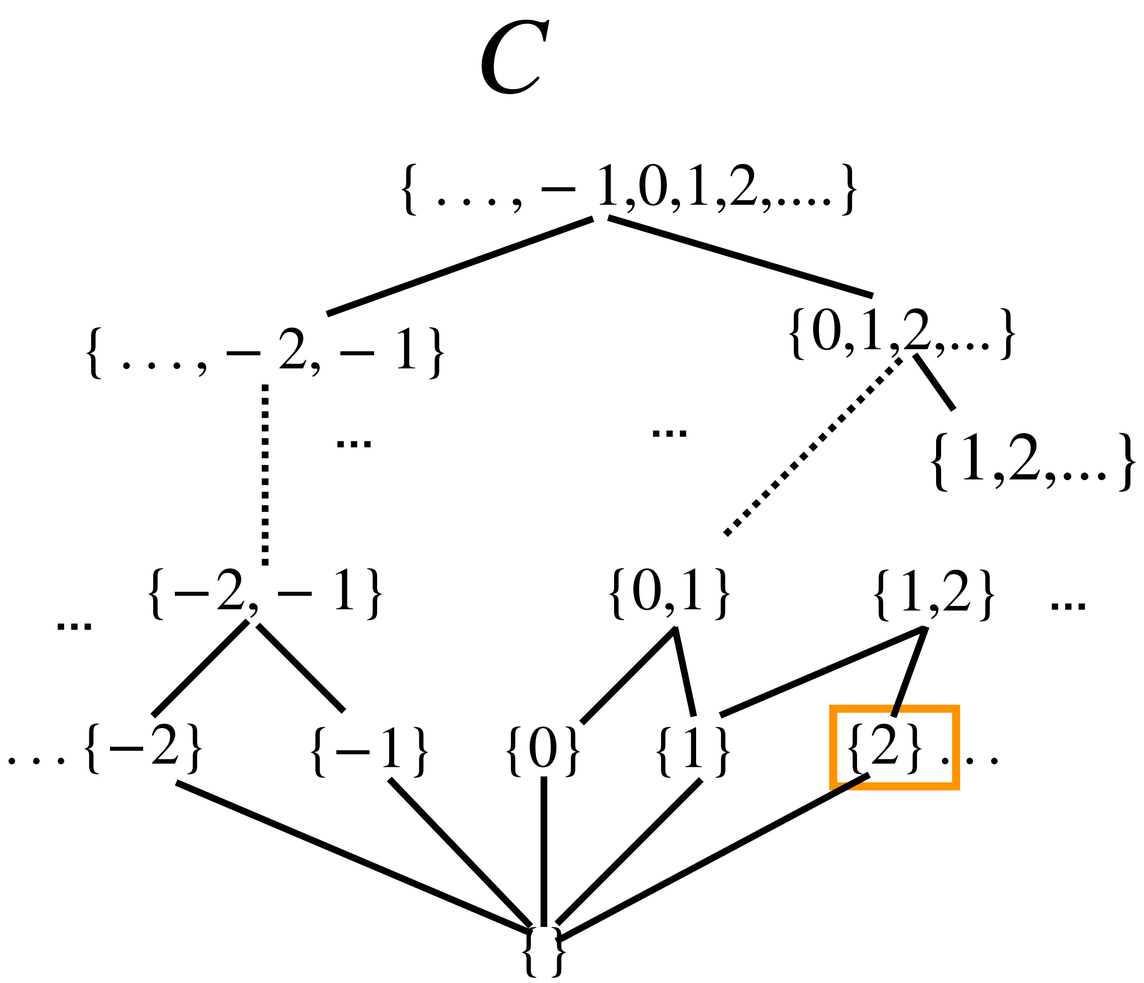
Safety verification problems

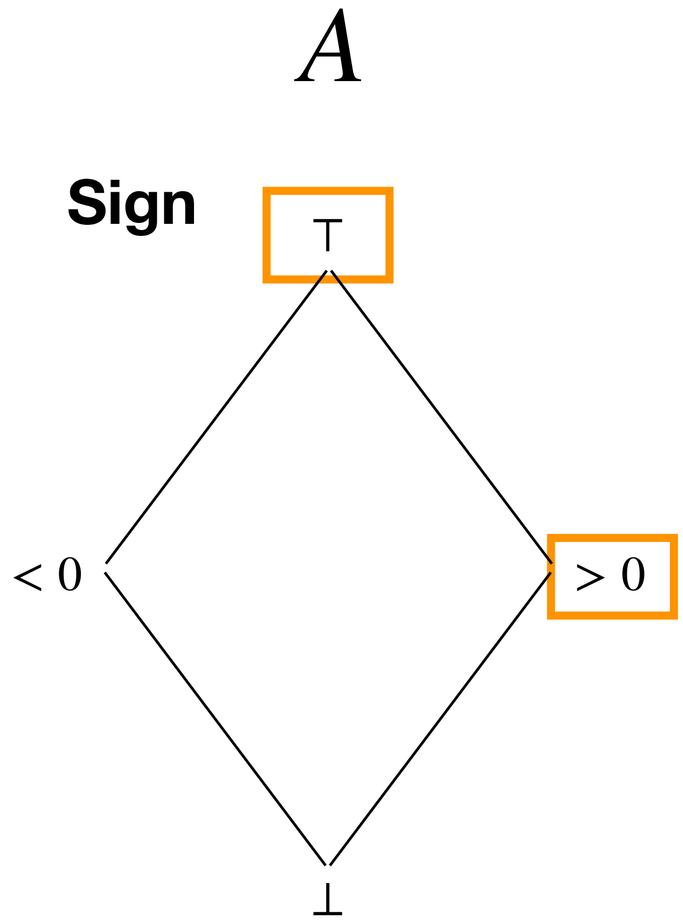
Given a specification S, then if:

- the specification S can be exactly represented in the abstract as $S^\sharp$, i.e., $S = \gamma(S^\sharp)$, and
- the result $P^\sharp$ of a static analysis is a sound abstraction of the concrete semantics P, i.e., $P \subseteq \gamma(P^\sharp)$ and
- $P^\sharp \sqsubseteq S^\sharp$ i.e., the analyzer proves in the abstract that the program satisfies its specification,

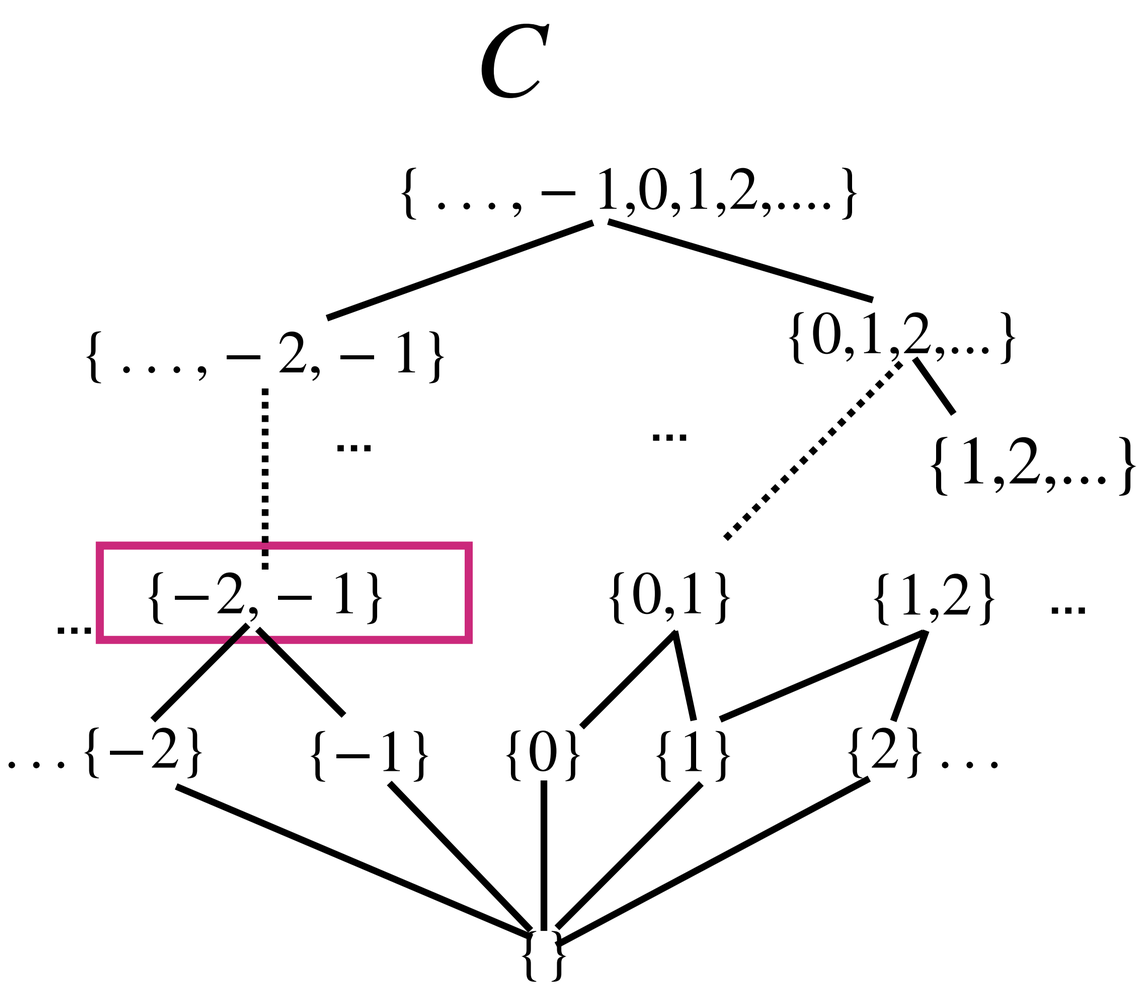
then, the program indeed satisfies its specification in the concrete, i.e., $P \subseteq S$. Note that, not only the computation of $P^\sharp$, but also the check $P^\sharp \sqsubseteq S^\sharp$, are done entirely in the abstract world. Hence, effective abstract algorithms lead to sound and effective static analyses.

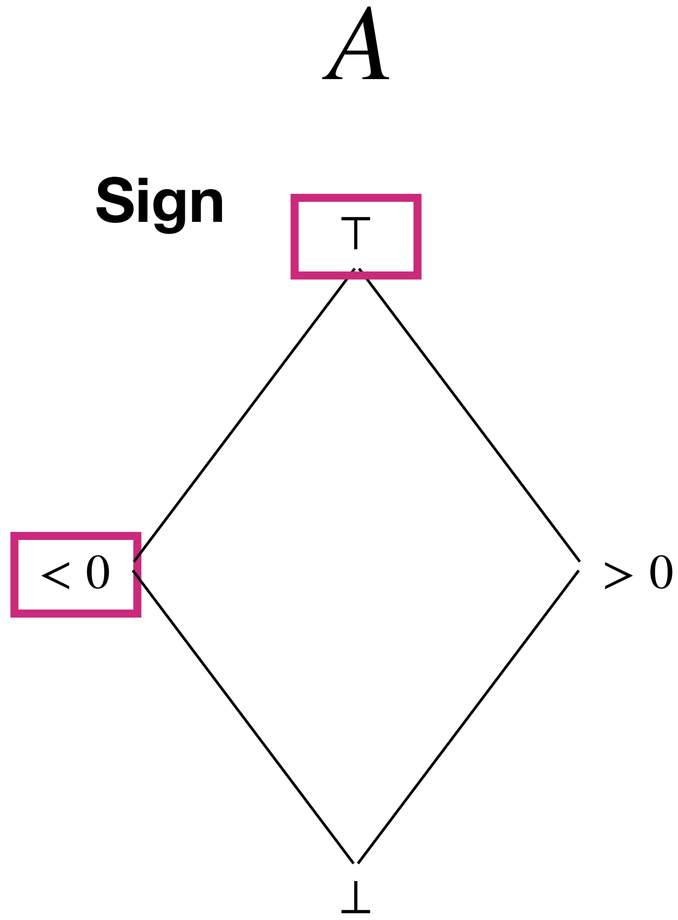
Sound abstraction



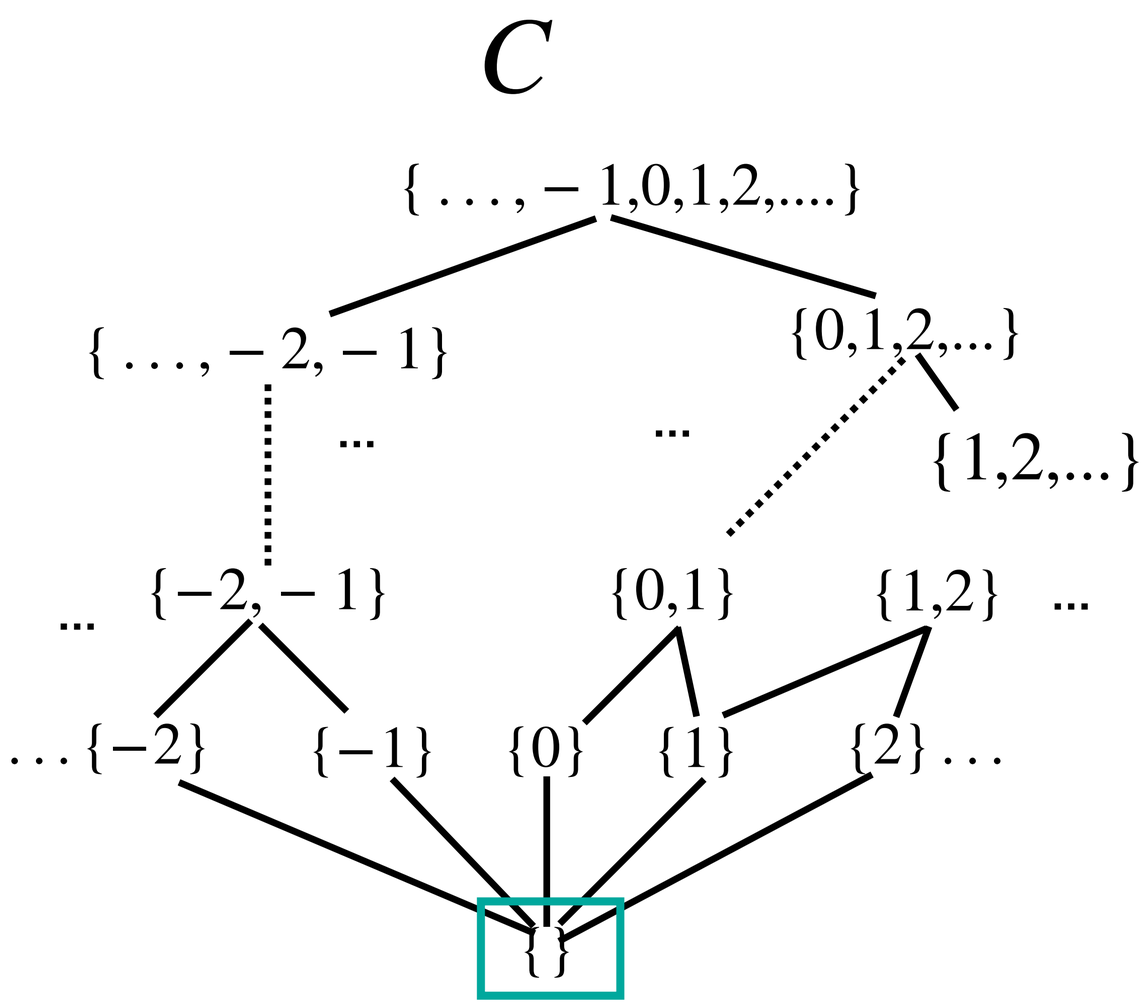


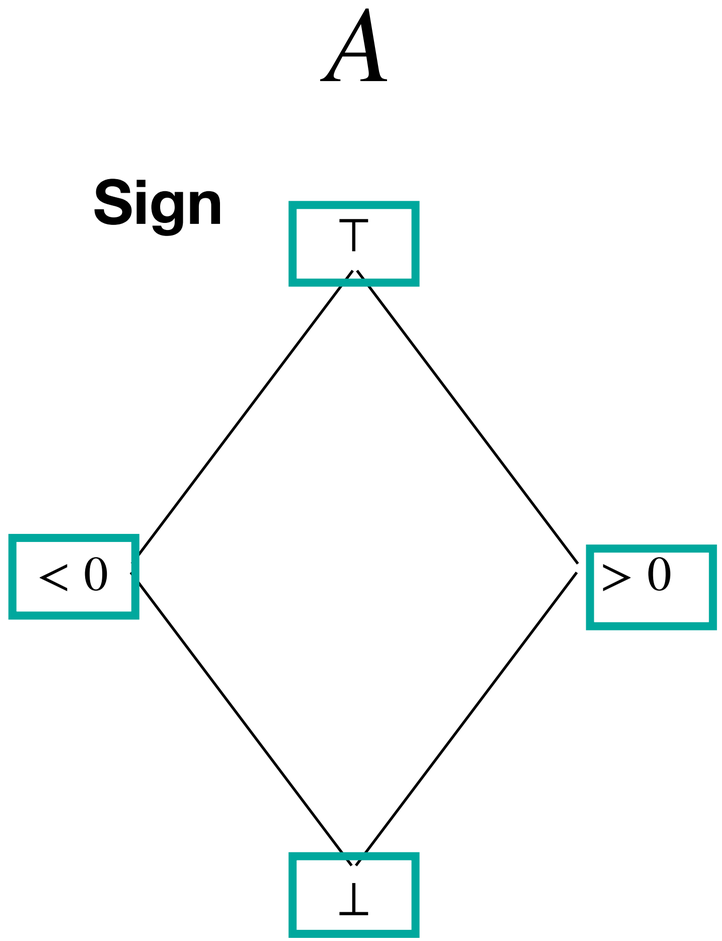
Sound abstraction



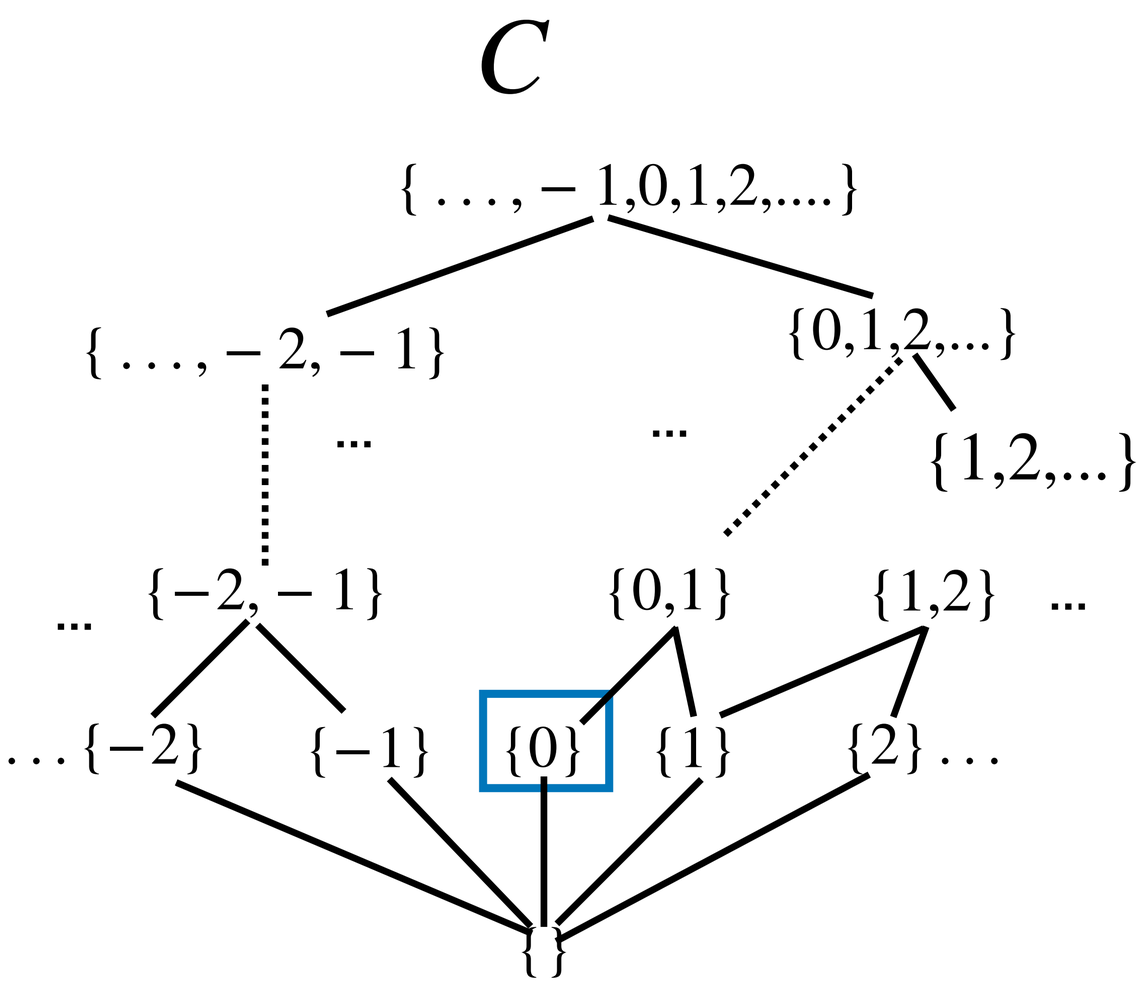


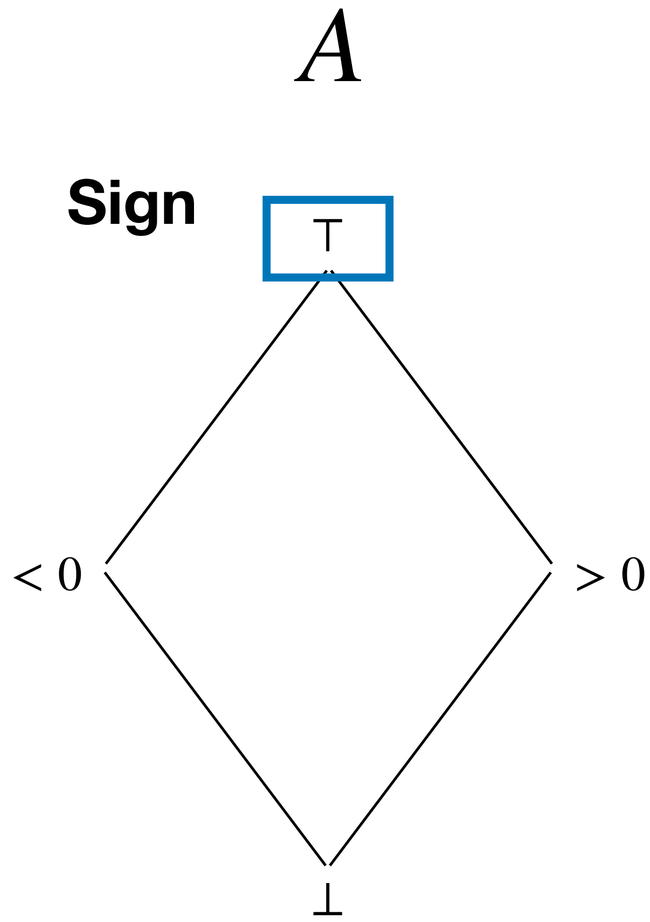
Sound abstraction



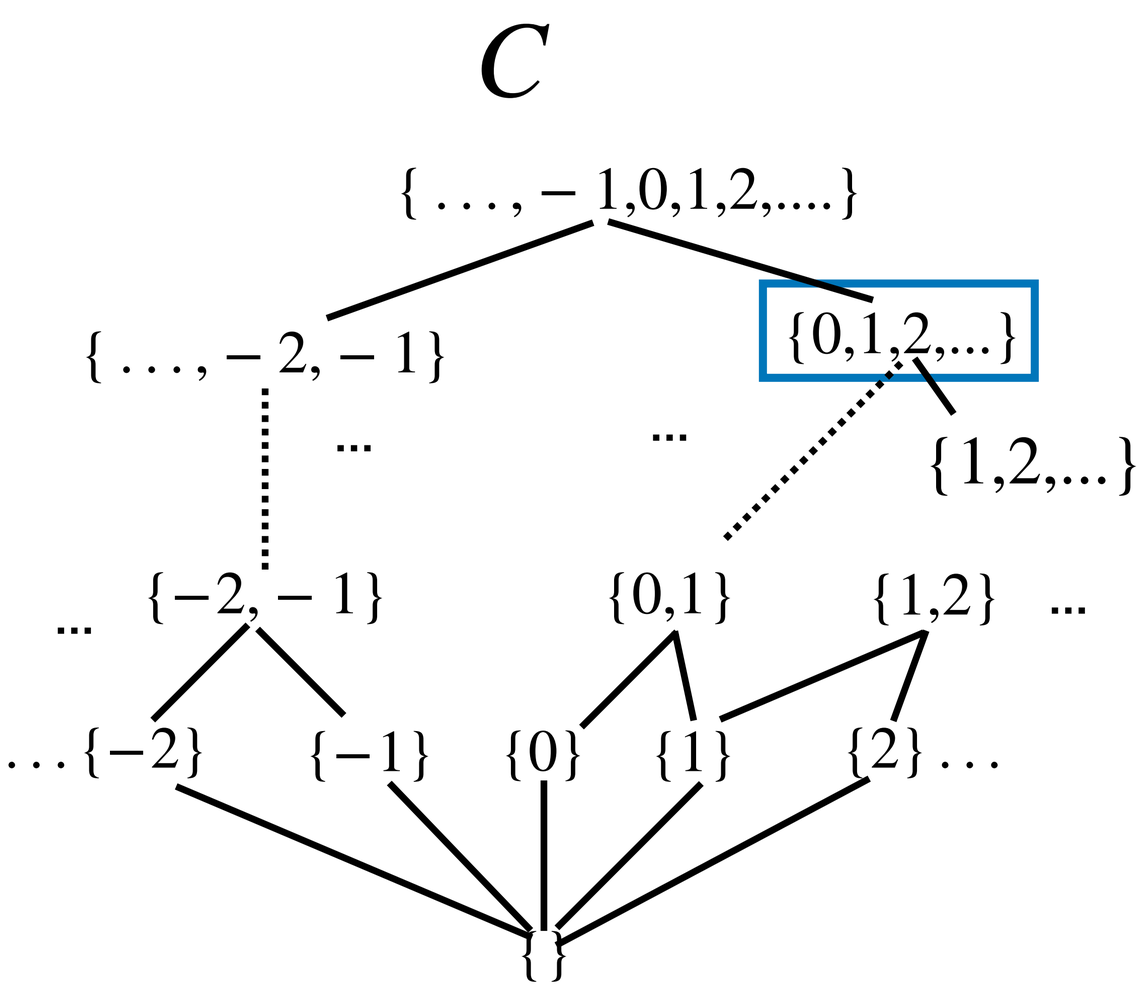


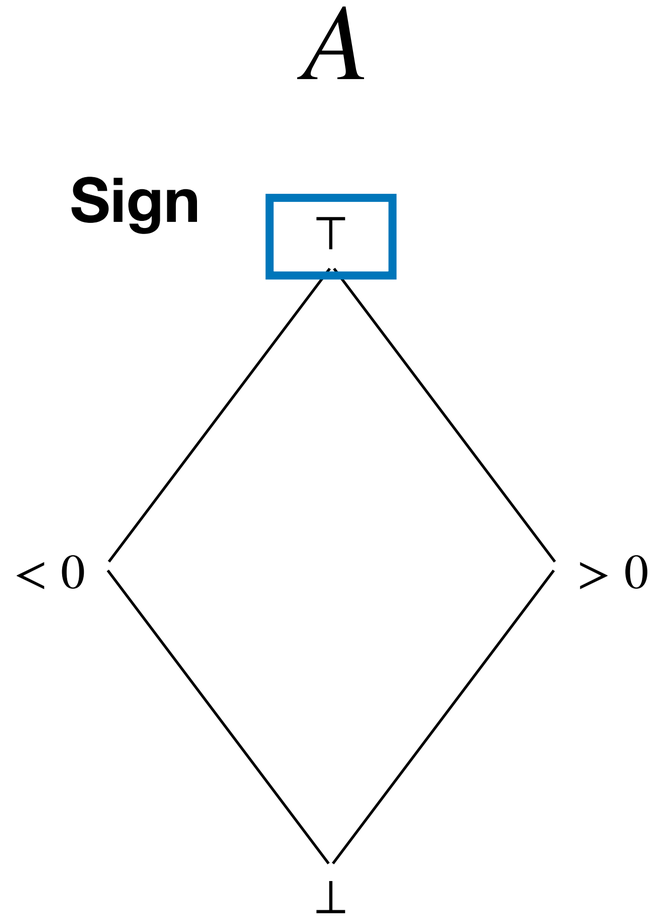
Sound abstraction





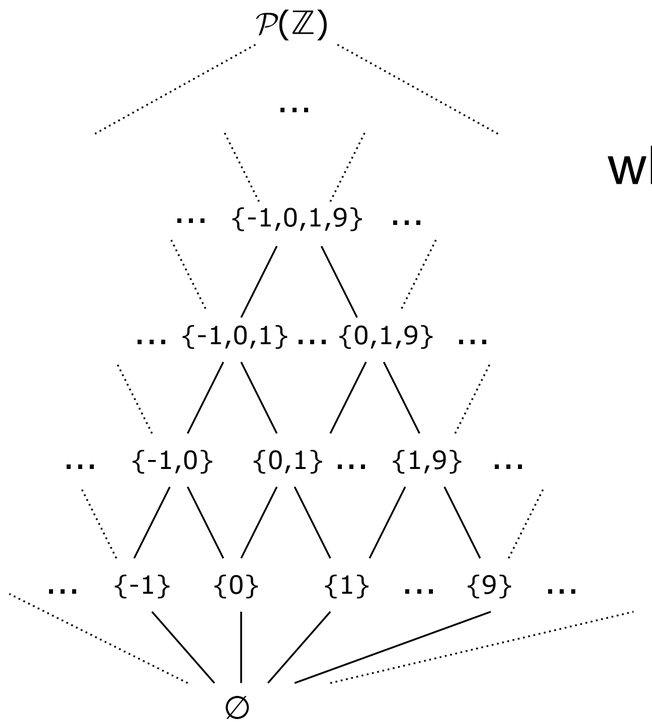
Sound abstraction





Abstraction

While a monotonic concretization γ is sufficient to reason about soundness, more structure, if available, can help us design sound and accurate analyses. In the standard Abstract Interpretation framework [Cousot and Cousot, 1977], we assume additionally the existence of a monotonic *abstraction function* $\alpha : C \to A$ that associates an abstract element to a concrete one such that (α, γ) forms a Galois connection:



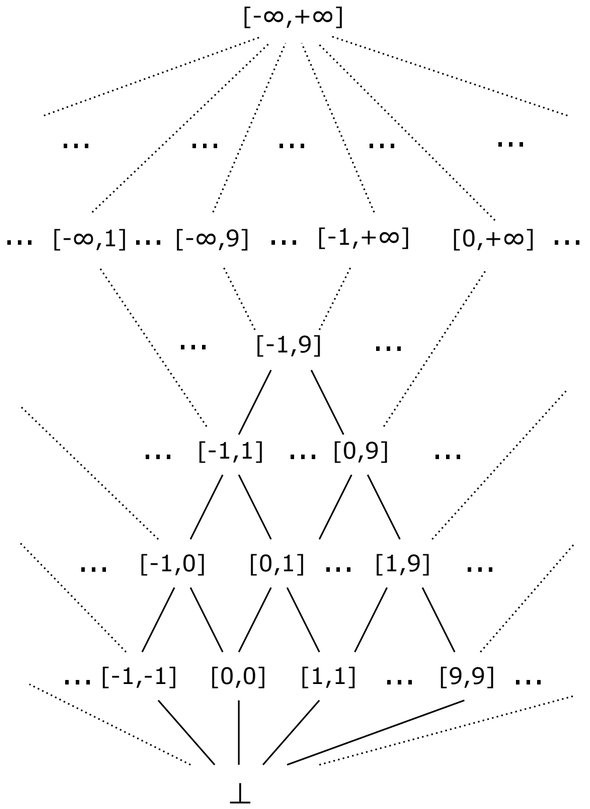
α

which intervals offer a sound abstraction for:

the finite set $\{0,5,9\}$?

the set of all powers of 2 ?

the set of all even numbers ?

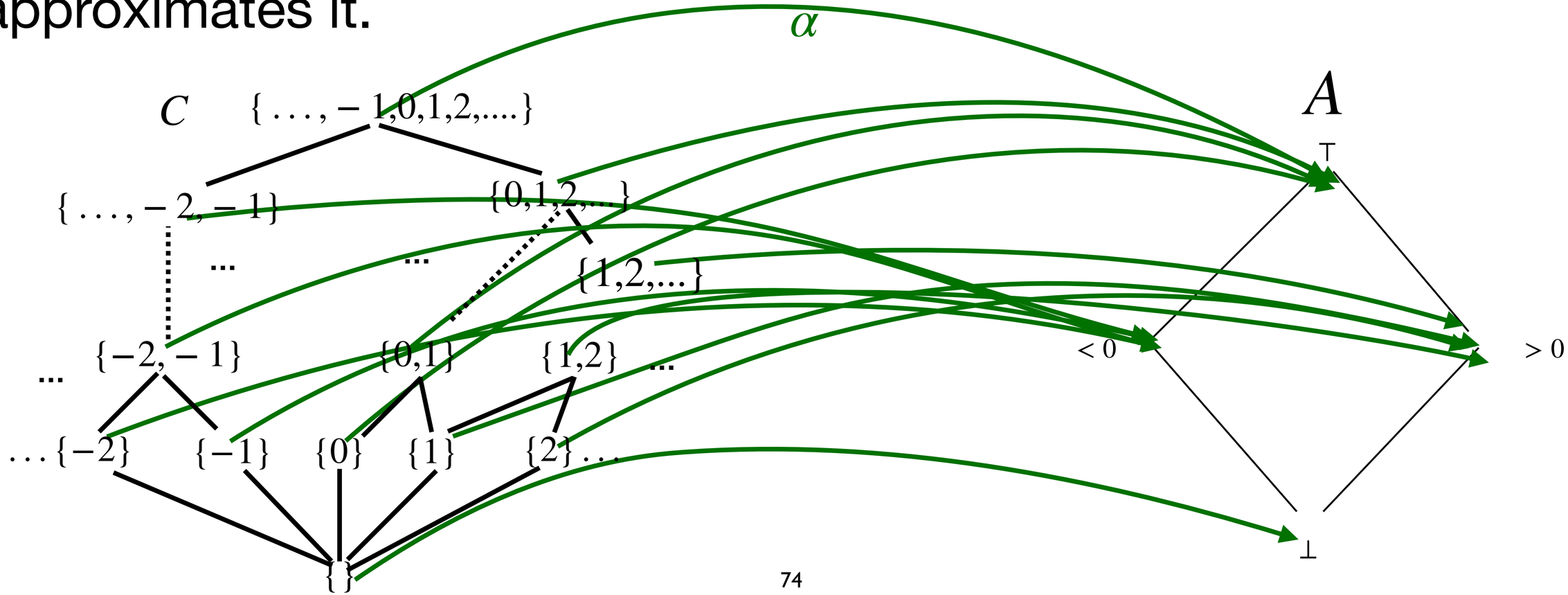


Abstraction function

Definition

Abstraction function $\alpha : C \rightarrow A$ is a monotone function

that maps c concrete into the most precise abstract element that approximates it.

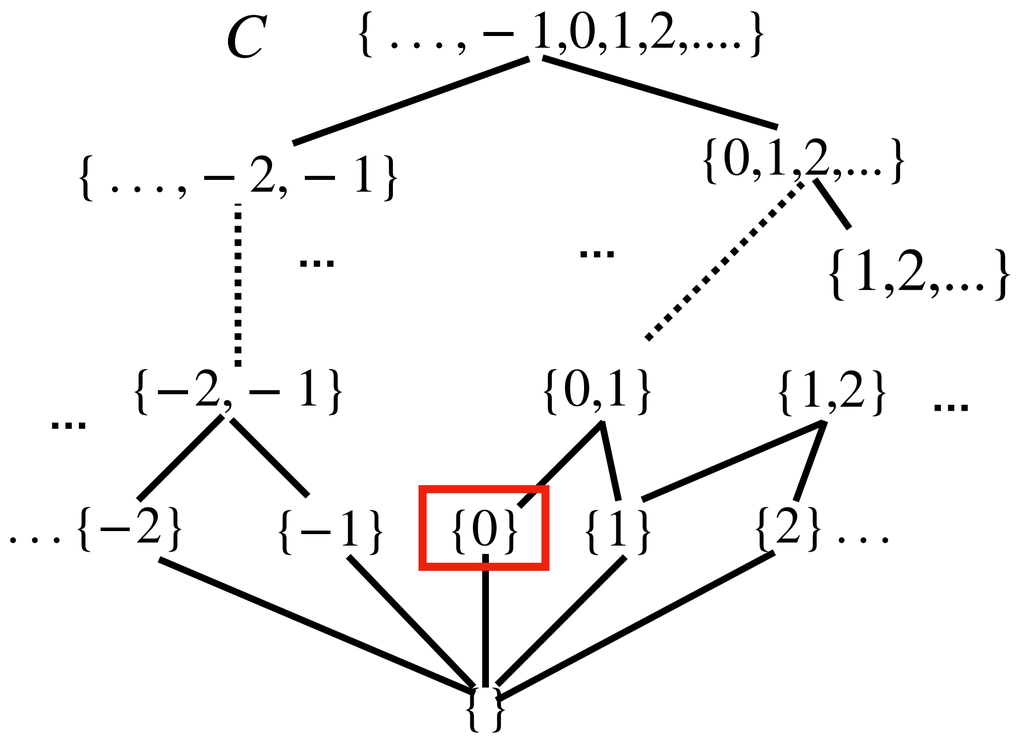


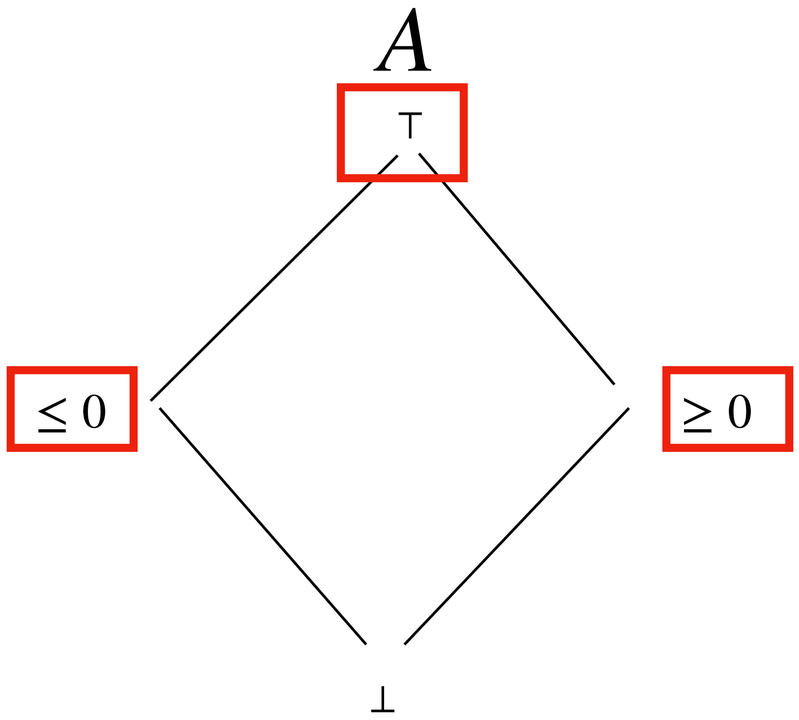
Abstraction function

Remark

To design an abstraction function $\alpha : C \rightarrow A$ the abstract domain must be closed under meet.

In this abstract domain the most precise element that approximates $\{0\}$ does not exist





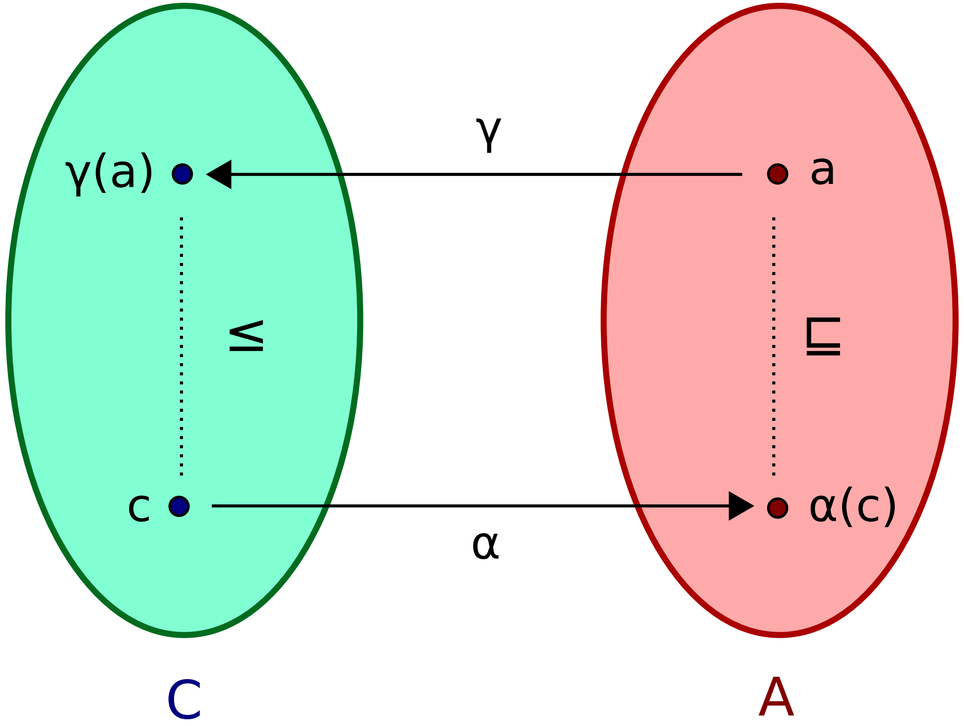
Galois connection and Galois insertion

Galois connection

Galois connection

From Wikipedia, the free encyclopedia

In mathematics, especially in order theory, a **Galois connection** is a particular correspondence (typically) between two partially ordered sets (posets). Galois connections find applications in various mathematical theories. They generalize the fundamental theorem of Galois theory about the correspondence between subgroups and subfields, discovered the French mathematician Évariste Galois.



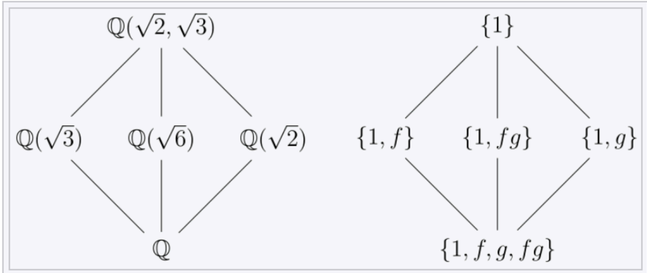
Galois theory

42 languages

Article Talk Read Edit View history Tools

From Wikipedia, the free encyclopedia

In mathematics, **Galois theory**, originally introduced by Évariste Galois, provides a connection between field theory and group theory. This connection, the fundamental theorem of Galois theory, allows reducing certain problems in field theory to group theory, which makes them simpler and easier to understand.



On the left, the lattice diagram of the field obtained from $\mathbf{Q}$ by adjoining the positive square roots of 2 and 3, together with its subfields; on the right, the corresponding lattice diagram of their Galois groups.

Galois introduced the subject for studying roots of polynomials. This allowed him to characterize the polynomial equations that are **solvable by radicals** in terms of properties of the permutation group of their roots—an equation is by definition *solvable by radicals* if its roots may be expressed by a formula involving only integers, *n*th roots, and the four basic arithmetic operations. This widely generalizes the Abel–Ruffini theorem, which asserts that a general polynomial of degree at least five cannot be solved by radicals.

Galois connections

Definition 2.12 (Galois connection)**.** *Given two posets* $(C, \leq)$ *and* $(A, \sqsubseteq)$*, the pair* $(\alpha : C \to A,\ \gamma : A \to C)$ *is a* Galois connection *if:*

$$\forall a \in A,\ c \in C, \boxed{c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a} \tag{2.5}$$

which is denoted as $(C, \leq) \underset{\alpha}{\overset{\gamma}{\leftrightarrows}} (A, \sqsubseteq)$.
α *and* γ *are said to be* adjoint functions, *where the abstraction* α *is the upper adjoint and the concretization* γ *is the lower adjoint.* ■

Definition 2.13. *A Galois connection* $(C, \leq) \underset{\alpha}{\overset{\gamma}{\leftrightarrows}} (A, \sqsubseteq)$ *is a* Galois embedding *if* $\boxed{\alpha \circ \gamma = id}$

Galois embeddings are also called Galois insertions

Must learn

Galois connection *if:*

$$\forall a \in A,\, c \in C,\, c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$$

Galois embedding *if* $\alpha \circ \gamma = id.$

Galois insertions

GC but not GI

γ is not injective ($\gamma(+) = \gamma(p)$)

α is not surjective ($p \notin \alpha(\wp(\mathbb{Z}))$)

$\gamma(\bot) = \varnothing$

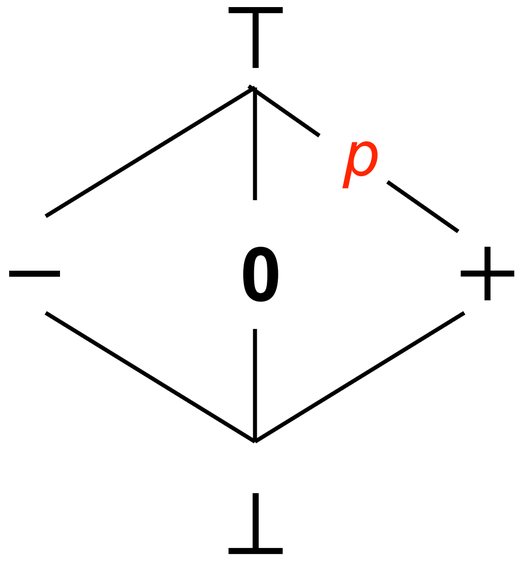
$\gamma(\mathbf{0}) = \{0\}$

$\gamma(-) =$ set of negative integers

$\gamma(+) = \gamma(p) =$ set of positive integers

$\gamma(\top) = \mathbb{Z}$

$\alpha(\{1,3,5\}) = +$



Abstract value *p* is "useless", because + offers a better abstraction

GIs from GCs

How to “repair” a GC into a GI by eliminating the “useless” abstract values?

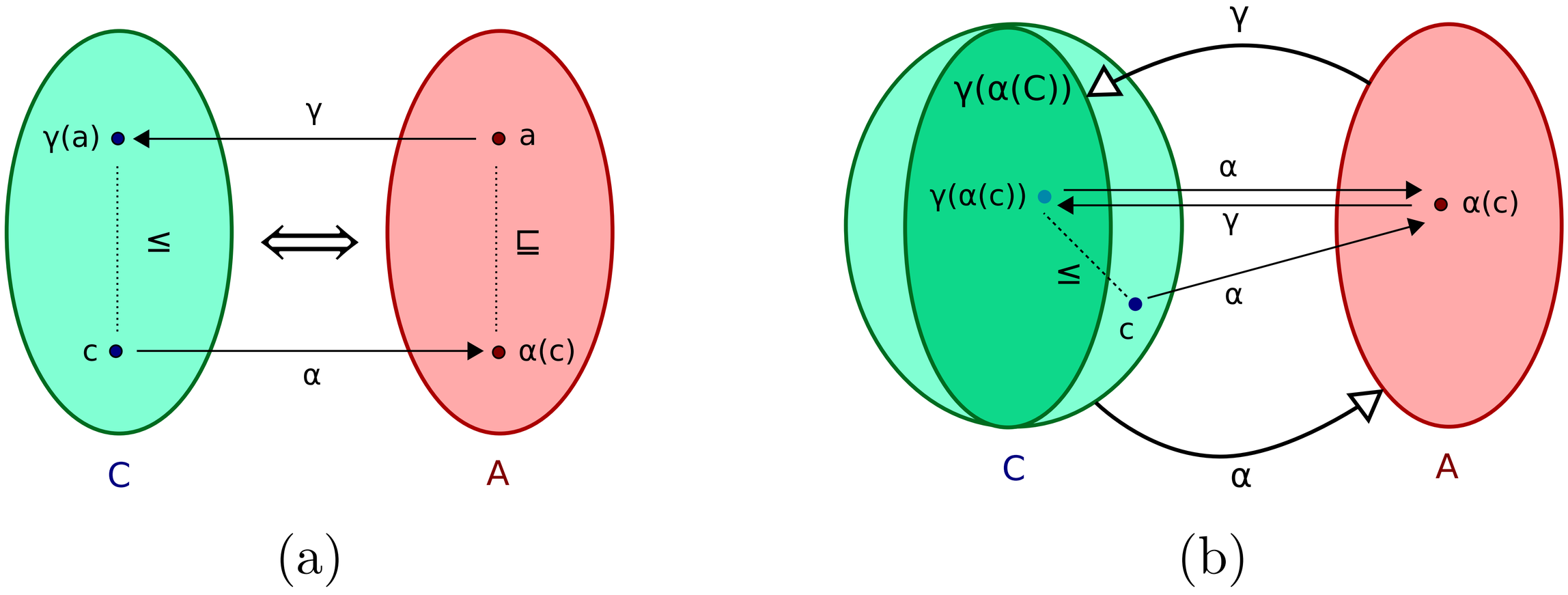
We take the image of C as a “reduced” abstract domain $A^{\text{red}} = \alpha(C)$ and restrict the original abstraction and concretization maps to operate on A^{red}:

$\alpha^{\text{red}} : C \to A^{\text{red}}$ with $\alpha^{\text{red}}(c) = \alpha(c)$

$\gamma^{\text{red}} : A^{\text{red}} \to C$ with $\gamma^{\text{red}}(a) = \gamma(c)$

We get a GI that has the same “expressive power” of the original GC

Galois connection



(a) (b)

Figure 2.8: A Galois connection (a) and a Galois embedding (b).

Example

Example 2.10 (Galois connection)**.** *Following up on Ex. 2.9.2, we define the Galois connection between the concrete domain* $\mathcal{P}(\mathbb{Z})$ *and the abstract domain of intervals represented as pairs of bounds (2.4):*

$$\{\,(a,b) \mid a \in \mathbb{Z} \cup \{-\infty\},\, b \in \mathbb{Z} \cup \{+\infty\},\, a \leq b\,\} \cup \{\,\bot\,\}$$

as follows:

$$\begin{array}{lcl} \gamma((a,b)) & \stackrel{\text{def}}{=} & \{\,x \in \mathbb{Z} \mid a \leq x \leq b\,\} \\ \gamma(\bot) & \stackrel{\text{def}}{=} & \emptyset \end{array}$$

$$\alpha(X) \stackrel{\text{def}}{=} \begin{cases} \bot & \text{if } X = \emptyset \\ [\min X,\, \max X] & \text{otherwise} \end{cases}$$

is it also a Galois insertion? 

i.e., does $\alpha \circ \gamma = \mathrm{id}$ hold?

Examples

1. *The interval Galois connection from Ex. 2.10 is actually an embedding.*

2. *Consider the following, alternate interval abstraction:*

$a \not\leq b$ $\not\perp$

$$\{\, [a,b] \mid a \in \mathbb{Z} \cup \{+\infty, -\infty\},\ b \in \mathbb{Z} \cup \{+\infty, -\infty\} \,\}$$

$[100,0] \sqsubseteq [9,6] \sqsubseteq [5,10]$

with order $[a,b] \sqsubseteq [c,d] \overset{\text{def}}{\iff} (a \geq c) \wedge (b \leq d)$ *and natural concretization:* $\gamma([a,b]) \overset{\text{def}}{=} \{\, x \in \mathbb{Z} \mid a \leq x \leq b \,\}$. *Then, we have* $\gamma([a,b]) = \emptyset$ *for any pair* $[a,b]$ *such that* $a > b$*: the concrete element* $\emptyset$ *has several representations in the abstract.*

We can construct an abstraction function α *as follows:* $\alpha(S) \overset{\text{def}}{=} [\min S, \max S]$, *so that* (α, γ) *is a Galois connection. However, it is not a Galois embedding. The abstraction* α *differs from that of the interval abstraction from Ex. 2.10 only for the empty set, where* $\alpha(\emptyset) = [+\infty, -\infty]$. *Note that, in order for the set of intervals* $\{\, [a,b] \mid a > b \,\}$ *representing the empty set to have a least element, we had to allow* $+\infty$ *as lower bound and* $-\infty$ *as upper bound; with only a slight extension of notations, we state that* $\min \emptyset = +\infty$ *and* $\max \emptyset = -\infty$. ♦

Absence of best abstraction

Example 2.11 (Absence of a Galois connection)**.** *Not all abstract domains enjoy a Galois connection. Consider the affine inequalities domain, that we mentioned in Sect. 1.1.2 and will present in details in Sect. 5.3. Intuitively, it abstracts a set of points as a convex polyhedron, and a best abstraction would be the smallest polyhedron enclosing these points. Some shapes, such as discs, do not have a smallest enclosing polyhedron, hence, no α function, and so, no Galois connection can exist. We can still reason about soundness through γ, though, and choose a, possibly arbitrary, way to soundly abstract a disc.* ♦

Exercises
They are all mandatory!

Alternative definition of GC

Definition 2.12 (Galois connection)**.** *Given two posets* $(C, \leq)$ *and* $(A, \sqsubseteq)$*, the pair* $(\alpha : C \to A, \gamma : A \to C)$ *is a* Galois connection *if:*

$$\forall a \in A,\, c \in C,\, c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a \tag{2.5}$$

The fundamental property of Galois connections (2.5) provides, in a very compact form, a strong connection between the concrete and the abstract world. The following theorem provides a less compact, but equivalent view:

Theorem 2.3 (Alternate characterization of Galois connections)**.** *We have a Galois connection* $(C, \leq) \underset{\alpha}{\overset{\gamma}{\leftrightarrows}} (A, \sqsubseteq)$ *if and only if the function pair* (α, γ) *satisfies all the following properties:*

1. γ *is monotonic;*
2. α *is monotonic;*
3. $\gamma \circ \alpha$ *is extensive, i.e.,* $\forall c \in C\colon c \sqsubseteq \gamma(\alpha(c))$*;*
4. $\alpha \circ \gamma$ *is reductive, i.e.,* $\forall a \in A\colon \alpha(\gamma(a)) \leq a$*.*

■

Ex. 1: Extensivity

Given two posets $(C, \leq)$ and $(A, \sqsubseteq)$ and two maps $\gamma\colon A \to C$ and $\alpha\colon C \to A$ such that $c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$, for all $c \in C$ and $a \in A$, prove that:

the map $\gamma \circ \alpha$ is extensive (i.e., that $c \leq (\gamma \circ \alpha)(c) = \gamma(\alpha(c))$ for all $c \in C$).

$\alpha(c)$ is a sound approx of c

Ex. 2: Reductivity

Given two posets $(C, \leq)$ and $(A, \sqsubseteq)$ and two maps $\gamma\colon A \to C$ and $\alpha\colon C \to A$ such that $c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$, for all $c \in C$ and $a \in A$, prove that: the map $\alpha \circ \gamma$ is reductive (i.e., that $\alpha(\gamma(a)) = (\alpha \circ \gamma)(a) \sqsubseteq a$ for all $a \in A$).

a is a sound approx of $\gamma(a)$

Ex. 3: Monotonicity

Given two posets $(C, \leq)$ and $(A, \sqsubseteq)$ and two maps $\gamma\colon A \to C$ and $\alpha\colon C \to A$ such that $c \leq \gamma(a) \iff \alpha(c) \leq a$, for all $c \in C$ and $a \in A$, prove that:

both maps γ and α are monotone .

Ex.4: Equivalent definition

Given two posets $(C, \leq)$ and $(A, \sqsubseteq)$ and two monotone maps $\gamma : A \to C$ and $\alpha : C \to A$ such that $\gamma \circ \alpha$ is extensive and $\alpha \circ \gamma$ is reductive prove that:

the maps α and γ form a Galois connection

(i.e., that $c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$, for all $c \in C$ and $a \in A$).

Properties of GC

Theorem 2.4 (Galois connection properties). *Given a Galois connection* $(C, \leq) \underset{\alpha}{\overset{\gamma}{\leftrightarrows}} (A, \sqsubseteq)$, *we have:*

1. $\gamma \circ \alpha \circ \gamma = \gamma$ *and* $\alpha \circ \gamma \circ \alpha = \alpha$; round trips
2. $\alpha \circ \gamma$ *and* $\gamma \circ \alpha$ *are idempotent;* idempotency
3. $\forall c \in C\colon \alpha(c) = \sqcap \{\, a \mid c \leq \gamma(a) \,\}$; γ induces α
4. $\forall a \in A\colon \gamma(a) = \vee \{\, c \mid \alpha(c) \sqsubseteq a \,\}$; α induces γ
5. α *maps concrete lubs to abstract lubs:*
   $\forall X \subseteq C$: *if* $\vee X$ *exists, then* $\alpha(\vee X) = \sqcup \{\, \alpha(x) \mid x \in X \,\}$; α preserves lubs
6. γ *maps abstract glbs to concrete glbs:*
   $\forall X \subseteq A$: *if* $\sqcap X$ *exists, then* $\gamma(\sqcap X) = \wedge \{\, \gamma(x) \mid x \in X \,\}$. γ preserves glbs ■

Ex.5: Round-trips

Given two posets $(C, \leq)$ and $(A, \sqsubseteq)$ and two maps $\gamma\colon A \to C$ and $\alpha\colon C \to A$ such that $c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$, for all $c \in C$ and $a \in A$, prove that:

both equalities $\gamma \circ \alpha \circ \gamma = \gamma$ and $\alpha \circ \gamma \circ \alpha = \alpha$ hold.

Ex.6: Idempotency

Given two posets $(C, \leq)$ and $(A, \sqsubseteq)$ and two maps $\gamma : A \to C$ and $\alpha : C \to A$ such that $c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$, for all $c \in C$ and $a \in A$, prove that:

the maps $\gamma \circ \alpha$ and $\alpha \circ \gamma$ are both idempotent .

Ex.7: γ induces α

Given two posets $(C, \leq)$ and $(A, \sqsubseteq)$ and two maps $\gamma\colon A \to C$ and $\alpha\colon C \to A$ such that $c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$, for all $c \in C$ and $a \in A$, prove that:
the map $\alpha\colon C \to A$ is such that $\alpha(c) = \sqcap \{\ a \mid c \leq \gamma(a)\ \}$ for all $c \in C$.

Ex.8: α induces γ

Given two posets $(C, \leq)$ and $(A, \sqsubseteq)$ and two maps $\gamma\colon A \to C$ and $\alpha\colon C \to A$ such that $c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$, for all $c \in C$ and $a \in A$, prove that:
the map $\gamma\colon A \to C$ is such that $\gamma(a) = \bigvee \{\ c \mid \alpha(c) \sqsubseteq a\ \}$ for all $a \in A$.

Ex.9: α preserves lubs

Given two posets $(C, \leq)$ and $(A, \sqsubseteq)$ and two maps $\gamma : A \to C$ and $\alpha : C \to A$ such that $c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$, for all $c \in C$ and $a \in A$, prove that: $\alpha(\vee D) = \sqcup \{ \alpha(c) \mid c \in D \}$ whenever the concrete lub $\vee D$ exists for $D \subseteq C$.

Ex.10: γ preserves glbs

Given two posets $(C, \leq)$ and $(A, \sqsubseteq)$ and two maps $\gamma : A \to C$ and $\alpha : C \to A$ such that $c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$, for all $c \in C$ and $a \in A$, prove that:
$\gamma(\sqcap B) = \wedge \{ \gamma(a) \mid a \in B \}$ whenever the abstract glb $\sqcap B$ exists for $B \subseteq A$.

Alternative definition of GI

Definition 2.13. *A Galois connection* $(C, \leq) \underset{\alpha}{\overset{\gamma}{\leftrightarrows}} (A, \sqsubseteq)$ *is a* Galois embedding *if any of the following, equivalent properties hold:*

1. α *is surjective:* $\forall a \in A : \exists c \in C : \alpha(c) = a$;

2. γ *is injective:* $\forall a, a' \in A : \gamma(a) = \gamma(a') \implies a = a'$;

3. $\alpha \circ \gamma = id$.

We denote a Galois embedding as $(C, \leq) \underset{\alpha}{\overset{\gamma}{\leftrightarrows\!\!\!\!\rightarrow}} (A, \sqsubseteq)$. ■

Properties 1 and 2 state that every abstract element is useful: there is no redundancy as every abstract element abstracts a different concrete element. Property 3 allows us to view the abstract domain A as isomorphic to a subset of the concrete domain C. This is depicted in Fig. 2.8.(b).

Ex.11: surjectivity and GI

Given two posets $(C, \leq)$ and $(A, \sqsubseteq)$ and two maps $\gamma : A \to C$ and $\alpha : C \to A$ such that $c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$, for all $c \in C$ and $a \in A$, prove that :

the maps $\alpha \circ \gamma = id$ iff α is surjective ($\forall a \in A \,.\, \exists c \in C \,.\, a = \alpha(c)$).

Ex.12: injectivity and GI

Given two posets $(C, \leq)$ and $(A, \sqsubseteq)$ and two maps $\gamma : A \to C$ and $\alpha : C \to A$ such that $c \leq \gamma(a) \iff \alpha(c) \sqsubseteq a$, for all $c \in C$ and $a \in A$, prove that :

the maps $\alpha \circ \gamma = id$ iff γ is injective ($\forall a, a' \in A \,.\, \gamma(a) = \gamma(a') \Rightarrow a = a'$).